NOVA SCHOOL OF SCIENCE & TECHNOLOGY

DEPARTAMENT OF COMPUTER SCIENCE

TIAGO SILVA MEIRIM

BSc in Computer Science and Engineering

# FROM OCAML TO CAKEML, AND BACK

## A PIPELINE FOR VERIFIED CODE BY CONSTRUCTION

Dissertation Plan
MASTER IN COMPUTER SCIENCE AND ENGINEERING

NOVA University Lisbon
July, 2025

NOVA SCHOOL OF SCIENCE & TECHNOLOGY

DEPARTAMENT OF COMPUTER SCIENCE

# FROM OCAML TO CAKEML, AND BACK

## A PIPELINE FOR VERIFIED CODE BY CONSTRUCTION

**TIAGO SILVA MEIRIM**

BSc in Computer Science and Engineering

**Adviser:** Mário José Parreira Pereira
*Assistant Professor, NOVA University Lisbon*

Dissertation Plan
MASTER IN COMPUTER SCIENCE AND ENGINEERING

NOVA University Lisbon
July, 2025

# Abstract

The study of the Formal Verification field is very important for critical systems that involve high levels of trust. A significant advancement in recent research has been the development of certified compilers, such as CakeML, which ensure that the generated machine code preserves the behaviour of the original program. The integration of these properties in a pipeline with previously verified code can provide powerful correctness guarantees.

The main objective of this work is to explore and develop a verification pipeline that starts with programs written in OCaml with GOSPEL annotations with the end goal of producing correct-by-construction CakeML code. The first step involves translating the annotated OCaml code to WhyML using the Cameleer tool. This WhyML code should then be verified on the Why3 platform and ultimately extracted to CakeML. Additionally, the pipeline should also feature a translation scheme in the opposite direction, in which we propose GOSPEL as a generic specification language for the ML family languages, in particular CakeML, in order to verify programs in these languages using Cameleer.

Currently, the extraction mechanism in Why3 to CakeML is outdated and only supports a subset of the language. As such, we intend to revisit and expand upon it in this work, by updating the translation scheme, implement a mechanism to stop the extraction of non-verified code and to report cases where translation is not possible due to incompatibilities between features. On the opposite direction we want to develop a translation scheme from CakeML to OCaml and reutilize the GOSPEL parser to allow verification with Cameleer.

In this document, a thorough study was conducted concerning the state of Why3's extraction mechanism, alongside the selected tools, to identify which steps of the pipeline require modifications. Moreover, we also present the theoretical concepts concerning Operational Semantics, Deductive Verification and Verified Compilers.

**Keywords:** Deductive Verification, Certified Compilers, Verification Pipeline, CakeML, Cameleer, GOSPEL, OCaml, Why3

# Resumo

O estudo da Verificação Formal é um campo muito importante para sistemas críticos que envolvem confiança de alto nível. Um avanço significativo na investigação recente tem sido o desenvolvimento de compiladores certificados, como o CakeML, que assegura que o código máquina gerado vai preservar o comportamento do programa original. A integração destas propriedades numa *pipeline* com código previamente verificado pode fornecer fortes garantias de correção.

O principal objetivo deste trabalho é explorar e desenvolver uma *pipeline* de Verificação onde começa com programas escritos em OCaml com anotações em GOSPEL, onde o objetivo final é produzir código CakeML correto por construção. O primeiro passo envolve a tradução de código OCaml anotado para WhyML através da ferramenta Cameleer. Este código WhyML deve ser posteriormente verificado na plataforma Why3 e em última instância extraído para CakeML. Adicionalmente, a *pipeline* deve também conter um esquema de tradução na direção oposta, onde propomos a utilização de GOSPEL como uma linguagem genérica de especificação para linguagens da família ML, em particular CakeML, de modo a verificar programas nestas linguagens utilizando a ferramenta Cameleer.

Atualmente, o mecanismo de extração do Why3 para CakeML é obsoleto e apenas suporta um subconjunto da linguagem. Como tal, tencionamos revisitar e expandí-lo neste trabalho, ao atualizar o esquema de tradução, implementar o mecanismo para parar a extração de código não verificado e reportar casos onde a tradução não é possível devido a incompatibilidades entre funcionalidades. Na direção oposta pretendemos desenvolver um esquema de tradução de CakeML para OCaml e reutilizar o parser de GOSPEL de modo a permitir verificação através de Cameleer.

Neste documento, foi conduzido um estudo minucioso sobre o estado do mecanismo de extração do Why3, em conjunto com as ferramentas escolhidas, para identificar quais os passos da *pipeline* requerem modificações. Além disso, apresentamos os conceitos teóricos sobre as Semânticas Operacionais, Verificação Dedutiva e Compiladores Verificados.

**Palavras-chave:** Verificação Dedutiva, Compiladores Certificados, *Pipeline* de Verificação, CakeML, Cameleer, GOSPEL, OCaml, Why3

# Contents

**List of Figures** vi

**Glossary** vii

**1 Introduction** 1
- 1.1 Motivation . . . 1
- 1.2 Problem Definition . . . 2
- 1.3 Goals and Expected Contributions . . . 3
- 1.4 Report Structure . . . 3

**2 Background** 5
- 2.1 Operational Semantics . . . 5
- 2.2 Hoare logic . . . 6
  - 2.2.1 Hoare Triples . . . 6
  - 2.2.2 Assignment Axiom . . . 6
  - 2.2.3 Rule of Composition . . . 7
  - 2.2.4 Weakest Pre-Condition Calculus . . . 7
- 2.3 OCaml . . . 7
  - 2.3.1 Immutability by default . . . 8
  - 2.3.2 ADTs (Algebraic Data Types) . . . 8
  - 2.3.3 First-Class and Higher-Order Functions . . . 9
  - 2.3.4 Side Effects in OCaml . . . 9
  - 2.3.5 Module System . . . 11
- 2.4 Why3 . . . 11
- 2.5 GOSPEL and Cameleer . . . 12
  - 2.5.1 Cameleer Example . . . 13

**3 State Of The Art** 16
- 3.1 Certified Compilers . . . 16
  - 3.1.1 CompCert . . . 17

3.1.2 CakeML . . . 17
3.1.3 PureCake . . . 19
3.2 Pipeline . . . 19

**4 Pipeline** **21**
4.1 Goal Pipeline . . . 21
4.2 Tool Modifications . . . 22
4.3 Case Studies . . . 23
4.3.1 Tail Recursive Factorial . . . 23
4.3.2 Exception recursive search . . . 24
4.3.3 Count Even For A List . . . 26
4.3.4 Depth-search tree . . . 27
4.3.5 Tree Comparison . . . 28
4.3.6 References . . . 30
4.4 Analyses For The Case Studies . . . 31

**5 Extraction** **32**
5.1 WhyML Subset . . . 32
5.2 Corrections to the CakeML Extraction Driver . . . 33
5.2.1 Capitalization Conventions . . . 33
5.2.2 Let Bindings Delimiters . . . 33
5.2.3 Exception Handling . . . 34
5.3 Summary and Pipeline Implications . . . 34

**6 Translation** **35**
6.1 Lexer . . . 35
6.2 Parser . . . 35
6.3 Abstract Syntax Tree(AST) . . . 36
6.4 GOSPEL Abstract Syntax Tree(GAST) . . . 36
6.5 AST To GAST . . . 36
6.6 Printer . . . 37

**7 Comparative Evaluation** **38**
7.1 Verification Guarantees and Trust Boundaries . . . 38
7.2 Comparative Case Studies . . . 39
7.2.1 Red Black Tree . . . 39
7.3 Results Comparison . . . 40

**8 Conclusion** **41**
8.1 Future Work . . . 41
8.2 Contributions . . . 41

**Bibliography** **42**

**Appendices**

**A Complete Case Studies** **49**
A.1 Red Black Trees . . . . . 57
A.1.1 Verified WhyML Code . . . . . 57
A.1.2 Generated CakeML Code From Extraction . . . . . 62
A.1.3 Generated CakeML Code + GOSPEL Annotations . . . . . 63
A.1.4 Generated WhyML Code . . . . . 67

# List of Figures

2.1 The Original Cameleer Pipeline [41] . . . . . 13
2.2 The Goals Proved in Why3 . . . . . 15

3.1 CompCert pipeline diagram [23, 1] . . . . . 17
3.2 CakeML multi-stage pipeline . . . . . 19
3.3 OCaml to WhyML pipeline with Cameleer . . . . . 20

4.1 Goal pipeline . . . . . 22

# Glossary

**CakeML** A functional programming language based on ML, designed with a formally verified compiler. It aims to provide a trustworthy foundation for building secure and reliable software, particularly in verified systems. *(pp. i, ii, iv–vi, 1–3, 5, 12, 17–25, 27–40, 50–57, 62, 63)*

**Cameleer** An automated deductive verification tool for OCaml programs using GOSPEL annotations. *(pp. i, ii, vi, 2, 3, 12, 13, 15, 19–22, 33)*

**CompCert** A realistic and verified compiler, which enables formal reasoning and machine-checked proofs of correctness for each compilation phase. *(pp. vi, 17–19)*

**GOSPEL** A specification language for the OCaml language, intended to be used in various purposes. The acronym stands for Generic OCaml SPEcification Language. *(pp. i, ii, iv, v, 1–3, 12–15, 20–24, 26, 27, 29–31, 35, 36, 38, 39, 49, 51, 53–56, 63)*

**OCaml** A Pragmatic functional programming language with roots in academia and growing commercial use. It supports highly complex features, such as generational garbage collection, type inference, parametric polymorphism, an efficient compiler, among many others. *(pp. i, ii, vi, 1–3, 5, 7–15, 17, 20–27, 29–32, 49, 51, 53–56)*

**Why3** A platform for deductive program verification that relies on external provers. *(pp. i, ii, vi, 2, 3, 7, 11–13, 15, 19–23, 25, 32, 38–40)*

**WhyML** The programming and specification language used in the Why3 platform. It contains many features commonly present in modern functional languages, and supports built-in annotations to verification purposes. *(pp. i, ii, iv–vi, 2, 11–14, 20–23, 32–40, 57, 67)*

# 1

# Introduction

## 1.1 Motivation

Progress in Formal Verification has steadily been made over the years. The foundational ideas emerged as early as 1937 by Alan Turing [51] and by 1949 a program represented as a flowchart was formally verified [36]. Other notable papers where mathematical propositions established purely through theoretical reasoning include works by Alan Turing [52], Church [7] and Gödel [54]. These results laid important groundwork in logic, computation, and formal reasoning without relying on physical implementation or empirical methods. Significant progress was made in the 1960s with the introduction of formal systems to reason about program correctness. Floyd's work on flowcharts [14] and Hoare's in programs [18] marked a shift toward systematic verification of program behaviour using logical assertions. In recent years, two major developments have transformed the landscape of formal verification. The first is the so-called SMT [44] (Satisfiability Modulo Theories) revolution, which has enabled powerful automated verification tools capable of discharging logical obligations without human intervention [4]. The second is the emergence of certified compilers, such as CompCert [28], which provably ensures that the generated machine code faithfully preserves the semantics of the source program.

Why are verified compilers such an important target for formal verification? If a verified program is compiled by a faulty compiler, the resulting executable may not preserve its intended behaviour, invalidating any higher-level correctness obtained previously. Compilers like CompCert and CakeML [24] address this issue by providing formally verified compilation pipelines [32, 16, 29], thereby eliminating a major source of uncertainty and ensuring that correctness is preserved from source code to machine code [27].

Developments in relation to verified code have become increasingly crucial to achieve correctness and provide safety, the way we define correctness has more than one definition, it can be specified informally or written in formal language [11]. The weight of functional languages for deductive software verification has been quite low despite having good candidates for verification, like OCaml. Ever since 2018 with the introduction of GOSPEL [6], a formal specification language designed specifically for OCaml, verification

has become easier with modular specification that doesn't need many changes to OCaml code. This wasn't the first time formal logic and proof has been merged with functional programming, previous and more foundational systems like Coq [40] and WhyML [12] have paved the way. With the development of a behavioural specification language for OCaml, soon after a deductive verification tool was created for this language, that being Cameleer [41], a tool that translates a formally-specified program into corresponding code in WhyML.

With the introduction of certified compilers and state-of-the-art automated verification tools, it is possible to merge them in a pipeline to create correct high-level programs that when compiled that are not faulty. This is a highly trustworthy deductive verification system, that benefits from the features and correctness guarantees from each component of the pipeline throughout all the steps. Our motivation stems from the insurance that such a pipeline could provide to the verification of OCaml programs with GOSPEL annotations that eventually compile to correct machine code, by using the CakeML compiler.

It is also important to distinguish two broad methodologies in deductive verification. On one hand, extraction-based verification involves developing verified programs directly within a proof assistant and then extracting executable code, ensuring correctness by construction. On the other hand, translation-based verification starts from existing codebases, often annotated with formal specifications, and translates them into an intermediate verification language. Verification conditions are then generated and discharged by automated or interactive theorem provers [29].

## 1.2 Problem Definition

In order to create a pipeline from OCaml to CakeML and back, there is the need to understand the already existing translation tools. Firstly, Cameleer takes as input OCaml programs with GOSPEL annotations and internally these are translated onto WhyML code, which can then be verified inside Why3's ide or extracted to another language. The Why3 extraction mechanism currently allows the translation from WhyML to four other languages, with one of them being CakeML. By combining these two mechanisms we reach our goal of creating a pipeline to produce compiled code completely verified. However, we observed that the current extraction mechanism concerning CakeML is severely outdated and unreliable, after extensive tests using multiple OCaml features since it failed to compile on several instances. Cameleer also features an extraction mechanism that currently produces OCaml code from its intermediary representation, WhyML, instead of producing CakeML code. Another facet of this pipeline is to translate CakeML programs into OCaml code, and surprisingly, there is no such tool. The development of this new tool should enable direct verification of CakeML code, allowing correctness to be established before any extraction process takes place. This leads us to the following question:

*How do extraction-based and translation-based approaches differ in ensuring program correctness?*

In our proposed pipeline, both extraction-based and translation-based approaches are integrated, offering a unique opportunity to examine how these methodologies apply for the same case studies. This dual exploration not only highlights the distinctions between the two, but also leads to reflection on:

1. *How do we bridge these tools to create a pipeline that ensures correctness throughout all steps?*
2. *How does adding a specification language affect the verification workflow of certified compilers?*

## 1.3 Goals and Expected Contributions

The first goal in this work is to have a detailed look into the features of both OCaml and CakeML languages, find the notable differences and absences, to know which features to test using the extraction mechanism. The next goal should be to change internally the source code of the Cameleer tool to support the extraction of CakeML code. With this, we should be able to identify in detail the issues with the translation, what are the necessary changes to compile correctly, which may include introducing new features or fixing syntactic issues for the features already supported by the extraction mechanism, directly in Why3 source code. For the features that are not supported by one of the languages, providing a comprehensive error message rather than allowing the extraction to take place. To make the pipeline more robust we should confirm if the program is verified beforehand when performing an extraction. Since, there is no translation mechanism from CakeML to OCaml, it creates the need to develop a new tool. Our objective is to extend the GOSPEL parser to support languages within the ML family, enabling the application of formal specifications directly to CakeML code. This will allow verification to take place prior to translation. Specifically, we aim to build a new translation tool, driven by a custom parser, that converts CakeML with GOSPEL annotations into OCaml with GOSPEL annotation. To achieve this, we define a subset of the CakeML code that respects all the syntactic and semantic differences, meanwhile flagging with an error message any unsupported features.

## 1.4 Report Structure

The document is divided by 5 chapters and 1 appendix.

In chapter 2 it is possible to find the foundational theoretical concepts for Hoare Logic and an introduction to the languages and tools used for verification and extraction mechanism.

Chapter 3 describes the relevant pieces of work already available on the context of certified compilers, while also mentioning the current verification pipeline available using Cameleer, Why3 and CakeML.

The appendix A contains the complete and ordered code used for each of the case studies.

# 2

# Background

In this chapter we present the necessary background for the formal verification field, operational semantics and the elected tools.

## 2.1 Operational Semantics

Operational semantics provide a formal framework to define the behaviour of programming languages by describing how programs execute step by step. Unlike denotational semantics, which maps programs to mathematical objects, or axiomatic semantics, which focuses on logical reasoning about program correctness, operational semantics are particularly well-suited for modelling concrete computation and the dynamics of program execution. A foundational style of this approach is Structural Operational Semantics (SOS), introduced by Plotkin [43], which defines program behaviour through a set of syntactic transition rules that describe how individual program constructs evolve during execution.

Operational semantics is often split into two main approaches: small-step (reduction-based) and big-step (natural) semantics. The small-step approach breaks down computation into a sequence of individual steps, making it useful for analysing intermediate program states, as well as properties like divergence or concurrency. In contrast, big-step semantics describes how entire expressions evaluate to final values in one go, which is more straightforward for reasoning about the overall behaviour of a program [31]. However, big-step semantics had a challenge when it came to handling non-terminating behaviour, so Leroy and Grall [31] introduced coinductive definitions to model both termination and divergence within a single framework. This innovation has played a key role in the development of verified compilers and proof systems like Coq [40].

In the semantics of modern programming languages, particularly those that integrate both functional and imperative paradigms, such as OCaml and CakeML, it becomes necessary to extend traditional operational semantics to accurately capture features like mutable state, exceptions and references [15].

## 2.2 Hoare logic

Hoare Logic is a formal system for reasoning about the correctness of computer programs. However, computer arithmetic often differs from the standard arithmetic familiar to mathematicians due to issues like finite precision, overflows, and machine-specific behaviours. To account for these differences, Hoare introduced a new logic [18] based on assertions and inference rules for reasoning about the partial correctness of programs. Drawing inspiration from mathematical axioms and formal proof techniques, he proposed a framework where program behaviour could be specified and verified using logical formulas. This laid the foundation for systematic program verification and emphasized the need to model computational constraints, such as those arising from the limitations of machine arithmetic, within a formal system.

In Cook's seminal work [9], Hoare Logic was significantly strengthened, since Cook presented a Hoare-style axiom system tailored to a simple programming language and rigorously established both its soundness and adequacy. He concluded that, under reasonable assumptions, Hoare Logic is not only intuitively effective but also formally complete as a system for reasoning about program correctness.

### 2.2.1 Hoare Triples

The main construction of Hoare logic is the *Hoare triple*, where $P$ is a pre-condition, $C$ is a program (or a fragment) and $Q$ is a post-condition:

$$\{P\}\ C\ \{Q\}$$

A Hoare triple expresses a partial correctness guarantee: if the pre-condition $P$ holds before executing a program fragment $C$, and if $C$ terminates, then the post-condition $Q$ will hold afterward. This is a partial correctness result since the termination of $C$ is not assured by the triple. Total correctness is achieved when termination is also guaranteed.

### 2.2.2 Assignment Axiom

$$\frac{}{\{P_0\}\ x := f\ \{P\}}\quad (assign)$$

Where $x$ is a variable identifier; $f$ is an expression; $P_0$ is obtained from $P$ by substituting $f$ for all occurrences of $x$.

The axiom expresses that to prove a post-condition $P$ holds after assigning the expression $f$ to the variable $x$, it suffices to prove the pre-condition $P_0$ before the assignment, where $P_0$ is obtained by substituting every occurrence of $x$ in $P$ with the expression $f$.

### 2.2.3 Rule of Composition

The inference rule for composition states that if the post-condition of the first program segment matches the pre-condition of the second, then the entire program will produce the intended result, assuming the initial pre-condition of the first segment holds.

$$\frac{P\ \{Q_1\}\ R_1 \qquad R_1\ \{Q_2\}\ R}{P\ \{(Q_1; Q_2)\}\ R} \quad (composition)$$

### 2.2.4 Weakest Pre-Condition Calculus

After setting an elegant axiomatic framework for reasoning about program correctness through the use of Hoare triples $\{P\}\, C\, \{Q\}$, its practical application in large-scale or automated verification tasks presents significant challenges. Chief among these is the burden of manually identifying appropriate pre-conditions and invariants. To address this, Dijkstra [10] introduced the weakest pre-condition calculus, which reformulates program correctness into a computational problem: given a command $C$ and a desired post-condition $Q$, the function $wp(C, Q)$ computes the weakest pre-condition $P$ such that $\{P\}\, C\, \{Q\}$ holds.

This transformation from proof obligations to a calculable pre-condition function represents a critical step toward automating program verification. Unlike Hoare's original formulation, which requires deductive reasoning to derive correctness properties, weakest pre-condition semantics allow for algorithmic generation of verification conditions, thereby enabling integration with automated theorem provers and SMT solvers. The influence of weakest pre-conditions is particularly evident in modern deductive verification tools such as Why3 [5], and Dafny [26].

## 2.3 OCaml

OCaml is a statically typed functional programming language rooted in the ML family, originally developed to serve as the implementation language for theorem provers such as LCF. It inherits the foundational principles of typed $\lambda$-calculus, formal logic, and abstract interpretation, and extends them through practical language design aimed at enabling both expressiveness and efficiency [13].

First released in the mid-1990s, OCaml is the principal evolution of the Caml dialect of the ML family. The name OCaml, originally short for Objective Caml, reflects the addition of object-oriented features to the Caml language. While Caml stood for Categorical Abstract Machine Language, OCaml moved away from its dependence on the original abstract machine model [30]. The language is primarily developed and maintained by INRIA, which continues to guide its implementation and evolution.

OCaml distinguishes itself from many academically inspired languages through its strong emphasis on performance. Its static type system eliminates the need for runtime

type checking by ensuring type correctness at compile time, thereby avoiding the performance overhead commonly associated with dynamically typed languages. This design enables OCaml to maintain high execution efficiency while preserving strong safety guarantees at runtime. Exceptions to this safety model arise only in specific low-level scenarios, such as when array bounds checking is explicitly disabled or when employing type-unsafe features like runtime serialization [13].

For the standard compiler toolchain features, OCaml has both a high-performance native-code compiler (ocamlopt) and a bytecode compiler (ocamlc). The native-code compiler produces efficient machine code via a sophisticated optimizing backend [33], while the bytecode compiler offers portability and rapid development. Both compilers are integrated with a runtime system that supports automatic memory management via a garbage collector and provides facilities for exception handling, concurrency, and system interaction.

### 2.3.1 Immutability by default

Immutability is promoted as a default design principle in this language. Rather than modifying existing values, new ones are created through expression evaluation. This absence of mutable shared state simplifies reasoning about program behaviour and eliminates many common sources of verification complexity, such as aliasing and unintended side effects.

```
let x = 5                                                    OCaml
let y = x + 1 (* x is not modified, just referenced *)
```

Since values are not modified in place, program semantics are preserved under substitution, facilitating referential transparency, making symbolic execution and logical reasoning over programs more straightforward in deductive verification.

### 2.3.2 ADTs (Algebraic Data Types)

In OCaml, data types fall into three broad categories: atomic predefined types (e.g., `int`, `bool`), type constructors provided by the language (e.g., `list`, `array`, `option`), and user-defined types, which are declared through the general mechanism of algebraic data types, for instance:

```
type 'a tree =                                               OCaml
  | Leaf
  | Node of 'a tree * 'a * 'a tree
```

A tree can either be a `Leaf`, which represents the empty tree, or a `Node`, which is a tuple with three elements, the subtree to the left, the value for the node and the subtree to the right, respectively.

ADTs support exhaustive pattern matching, which is particularly useful for enabling structural recursion and inductive reasoning. From a verification perspective, this ensures

all possible cases are covered, making formal reasoning both precise and complete. One example of pattern matching combined with ADTs is:

```
let rec cmp t1 t2 =
  match t1, t2 with
  | Leaf, Leaf -> true
  | Leaf, _ -> false
  | _, Leaf -> false
  | Node (l1,x1,r1), Node (l2,x2,r2) -> cmp l1 l2 && x1 = x2 && cmp r1 r2
```

*OCaml*

The function `cmp` uses exhaustive pattern matching to compare structurally the two trees. When a tree is a leaf it means it is empty, as such, two empty trees are always equal. By contrast if one is a leaf, but the other is not, then they are different, since a node represents a non-empty tree. Finally, if both are nodes then it depends on the respective root and subtrees, in which case we have to recursively call the function `cmp` for the right and left subtrees.

### 2.3.3 First-Class and Higher-Order Functions

OCaml treats functions as first-class values: they can be passed as arguments, returned from other functions, and stored in data structures. Combined with lexical scoping and immutable data, this makes OCaml particularly well-suited for working with higher-order abstractions, a feature that aligns naturally with formal systems based on higher-order logic.

```
let apply_twice f x = f (f x)
let square x = x * x
let result = apply_twice square 2  (* returns 16 *)
```

*OCaml*

In the context of deductive verification, such functional abstractions support elegant formulations of parametric specifications and reasoning principles.

### 2.3.4 Side Effects in OCaml

OCaml is not a purely functional language since it allows the use of imperative constructs with side effects, like references, exceptions, built-in arrays, for and while loops. References allow the manipulation of memory, a feature not common for functional languages. A simple example about Euclidean division is displayed below:

```
let rec eudiv_aux x y r q =
  if !r >= y then (r := !r - y; q := !q + 1; eudiv_aux x y r q)
  else ()

let euclidean_div x y =
    let r = ref x and q = ref 0 in
  eudiv_aux x y r q;
  (!q, !r)
```

*OCaml*

The main function `euclidean_div` receives the two arguments `x` and `y`, which represent respectively the dividend and divisor. This function initializes the remainder `r` as a reference with the value of the dividend `x` and the quotient `q` as a reference of value 0. Since these variables are references, the auxiliary function may alter their in-memory value and by the end of its execution, the references `q` and `r` point to the correct values.

This example has the need for an auxiliary function, `eudiv_aux`, which receives as arguments the dividend `x`, the divisor `y`, the remainder `r`, and the quotient `q`. This function recursively increments the quotient by 1 and subtracts the remainder by the divisor, until the remainder is a non-negative value that is strictly smaller than the divisor, meaning it can not be subtracted any further, and therefore the process is finished. The values for the remainder and quotient are saved inside references and reassigned after every iteration.

OCaml also provides exception handling in a structured way to deal with runtime errors by allowing programmers to define and raise custom exceptions. In addition to user-defined exceptions, OCaml provides a range of built-in exceptions, which are extensively used by the standard library to handle common error cases, for example `Not_found`. Also, it is important to mention that built-in arrays are mutable have a fixed length and allow direct access to elements using their index. We combine these two elements on the example below:

*OCaml*

```
let rec search_aux a c n =
  let exception Break of int in try
    if c = Array.length a then raise Not_found
    else if a.(c) = n then raise (Break c)
    else search_aux a (c+1) n
  with Break i -> i

let linear_search a n = search_aux a 0 n
```

The core function, `linear_search`, delegates the search process to the auxiliary function and initializes it with the current index as 0, as to ensure it starts at the beginning of the array. The auxiliary function `search_aux` receives as arguments an array, the current index and a value that is going to be compared to each element of the array. This function recursively traverses the array, and returns the index of the array in case the desired value was found early by raising the exception `Break`, and catching it within the `try` block. By contrast, if the value is not in the array the exception `Not_found` is raised at the end but not caught.

Overall, while OCaml encourages a functional programming style and includes strong support for immutability and higher-order functions, its imperative features make it a multi-paradigm language that combines the benefits of both functional and imperative styles.

### 2.3.5 Module System

The module system enables parametric modularity through modules and functors, supporting abstraction, separation of concerns, and scalable design. At the heart of this system are signatures, which serve as formal interfaces specifying the types and values a module must provide while hiding the implementation details. These signatures play a key role in formal verification by allowing reasoning about components based solely on their interfaces, without depending on how they are implemented.

This modular structure is especially useful in verification contexts because it clearly separates the interface from the implementation. This separation makes it easier to reason about and verify each component independently, improving maintainability and correctness.

We can expand on the example found in the section 2.3.2 by encapsulating a module around it:

```
module Tree = struct                                                    OCaml

  type 'a tree =
    | Leaf
    | Node of 'a tree * 'a * 'a tree

  let rec cmp t1 t2 = (* ... *)
end
```

With this example we have created our own minimal `Tree` library, which includes the definition of a binary tree and the equality operation through the `cmp` function.

## 2.4 Why3

Why3 is the successor to the Why verification platform, offering a rich first-order language and a highly configurable toolchain for generating proof obligations in multiple formats. Its development is driven by the need to model both purely functional and imperative program behaviours and to formally verify their properties. Since verifying non-trivial programs typically requires abstracting them into pure logical models, Why3 is designed to bridge the gap between practical programming constructs and formal reasoning frameworks.

Why3 introduces WhyML, a specification and programming language that serves both as an expressive front-end and as an intermediate language for verifying programs written in other languages such as C, Java, and Ada. [12] It supports rich language features including pattern matching, recursive definitions, algebraic data types, and inductive or coinductive predicates. Moreover, it comes with a standard library of logical theories covering arithmetic, sets, maps, and more.

Why3 sets itself apart from other approaches that also provide rich specification languages like Coq [40] and Isabelle [20] by aiming to maximize automation. Rather than

functioning as a standalone theorem prover, it serves as a front-end that generates proof obligations to be discharged by external automated provers such as Z3 [37], Alt-Ergo [8], and cvc5 [3], as well as interactive systems like Coq and Isabelle.

In the context of automated program verification, Why3 simplifies the process by automatically generating verification conditions from annotated source code and delegating their resolution to a variety of powerful external theorem provers. When certain features (e.g., polymorphic types or pattern matching) are not supported by a backend prover, Why3 automatically applies transformations to encode them into a compatible form. This architecture allows developers to focus on writing correct specifications while benefiting from automation in proving correctness properties. [5]

## 2.5 GOSPEL and Cameleer

The evolution of deductive verification and proof automation has progressed significantly over the years. Recently, a combination of tools has been developed to apply these technologies to OCaml, a language that has not been widely explored in this context [41]. The need for a verification framework was addressed with the introduction of Cameleer, a tool for the deductive verification of programs written in OCaml, whose main objective is the automatic proof of functional code. However, Cameleer alone, using only standard OCaml, cannot perform these proofs. To enable verification, GOSPEL specifications must be added to the OCaml code.

GOSPEL terms are defined using the semantics of Separation Logic and are applied to OCaml interfaces. The acronym GOSPEL stands for "Generic OCaml Specification Language," indicating that the specification logic is not limited to a single tool but is intended for a variety of uses, such as verification, testing, and informal documentation [6]. GOSPEL includes a built-in parser designed primarily for OCaml, but due to the syntactic similarities, it can also be adapted or extended to parse subsets of related languages such as CakeML. Unlike other behavioural specification languages such as SPARK [47] and JML [25], GOSPEL supports Separation Logic, a significant extension of Hoare Logic and a powerful framework for reasoning about real-world programs [45, 39]. While GOSPEL is not the first tool to use Separation Logic, it uniquely introduces implicit permission association in data types, a feature not found in tools like VeriFast [19] or Viper [38] [6].

Cameleer takes as input an OCaml program annotated with GOSPEL specifications and translates it into WhyML, the intermediate language used by the Why3 platform. Once translated, the code can be processed by Why3, which leverages a range of automated theorem provers, such as Alt-Ergo [8], Z3 [37], and cvc5 [3], to discharge verification conditions. This seamless integration between Cameleer, GOSPEL, and Why3 significantly enhances the level of proof automation, allowing developers to verify functional correctness properties of OCaml programs with minimal manual intervention. This behaviour is captured by the following pipeline:

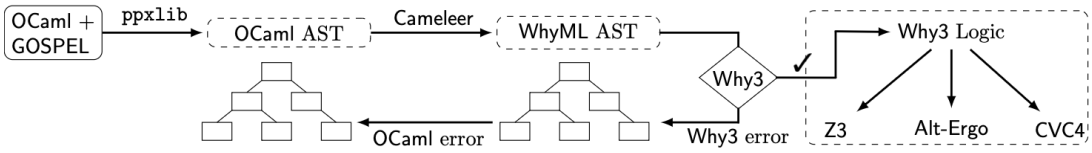


Figure 2.1: The Original Cameleer Pipeline [41]

Figure 2.1 demonstrates how the translation mechanism integrates with surrounding frameworks to produce deductively verified OCaml code annotated with GOSPEL specifications. If an error is detected in the original OCaml code during translation, the process reverts to the source, requiring corrections to ensure that the generated WhyML code is syntactically and semantically valid. Within the Why3 tool, if the generated verification goals cannot be discharged by the automated provers, it indicates that the specifications may be imprecise or incomplete and require refinement. These issues are highlighted by the tool, guiding the user to improve the annotations. Once all verification conditions are successfully proven, the resulting program is guaranteed to respect its specification, ensuring a high level of formal correctness [11].

The Cameleer pipeline in figure 2.1 follows the translation-based paradigm, leveraging WhyML as the intermediate language for verification.

### 2.5.1 Cameleer Example

Let us take this simple example for a recursive factorial in OCaml + GOSPEL after applying the Cameleer tool:

*GOSPEL + OCaml*

```
let rec fact_aux n c t =
  if c <= n then fact_aux n (c+1) (t*c) else t
(*@
  r = fact_aux n c t
  requires n >= 0
  requires 0 < c <= n+1
  requires t = factorial (c-1)
  ensures r = factorial n
  variant n+1-c
*)
```

The auxiliary function `fact_aux` receives as arguments the `n` value, that represents the factorial we want to compute, the `c` value, that represents the current index of the iteration, and `t` value, that is accumulating the result from previous iterations, in order to display the solution.

The specifications include a few pre-conditions, from those, the factorial to compute is higher or equal to 0, because the factorial as default is only calculated for non-negative values, the index of the iteration is between 0 and factorial to compute, because from the code written when the index achieves the same value as the factorial to compute the function should terminate. For the post-conditions we only need to make sure the result

of the auxiliary function is indeed the factorial of n, which is calculated logically with the definition below. The variant serves to prove the termination of the recursive function with an expression that decreases every iteration and is limited by 0. In this case since the counter is approaching n+1 every iteration, for the last iteration t holds the factorial of n, which was the previous value of c and terminates successfully.

In the specification of function `fact_aux` we use the logical definition of the factorial function which we present below:

*GOSPEL + OCaml*

```
(*@
  function rec factorial (n:int) :int =
    if n=0 then 1 else n * factorial (n-1)
*)
(*@
  requires n >= 0
  variant n
*)
```

This GOSPEL function represents the logical concept of a factorial, this can be used in other specifications to compare results. The implementation of this function is in a recursive way, since GOSPEL only supports the functional paradigm. Additionally, logical definitions are meant to be simple rather than complex but optimized, since we want to simplify the proof.

Finally, the main function `fact`:

*GOSPEL + OCaml*

```
let fact n = fact_aux n 1 1
(*@
  r = fact n
  requires n >= 0
  ensures r = factorial n
*)
```

Initializes the auxiliary call with appropriate base values, the first one is the counter so it needs to start at 1 because for the multiplication with the accumulator starting at 0 would not take into consideration the first iteration. For the 1 in the accumulator, the third argument, represents the neutral element for multiplication. The GOSPEL annotations specify the expected behaviour, including pre-conditions such as non-negativity of `n` and the correctness of the returned result.

The translated code to WhyML creates 3 goals, one for the function `fact` and another for the logical function `factorial`. After splitting the goals we get for `fact` the variant and pre-condition, and for `factorial` the invariant initialization, the invariant preservation, the post-condition and the VC:

| Status | Theories/Goals | Time |
|---|---|---|
| ✓ | factorial_rec.ml | |
| ✓ | Factorial_rec | |
| ✓ | factorial'vc [VC for factorial] | |
| ✓ | Z3 4.14.0 | 0.02 (steps: 95499) |
| ✓ | fact_aux'vc [VC for fact_aux] | |
| ✓ | Alt-Ergo 2.6.0 | 0.04 (steps: 128) |
| ✓ | fact'vc [VC for fact] | |
| ✓ | Z3 4.14.0 | 0.02 (steps: 97398) |

Figure 2.2: The Goals Proved in Why3

All verification goals were successfully discharged in figure 2.2 through the automated provers, resulting in the original OCaml code, annotated with GOSPEL specifications, being fully verified. This outcome demonstrates the effectiveness of the verification pipeline, where the combination of Cameleer and Why3 enables high levels of automation in proving the functional correctness of OCaml programs.

# 3

# State Of The Art

The State-Of-The-Art outlines the most relevant and advanced work currently available in the area covered by this work. It provides an overview of existing research, tools, and methodologies, helping to frame the context in which this work is situated. By reviewing what has already been accomplished, this section highlights ongoing challenges and uncovers the gaps that this work aims to address.

## 3.1 Certified Compilers

A compiler is a software system that translates a program written in a source programming language into an equivalent representation in a target language, typically a lower-level language such as assembly or machine code. The goal of a compiler is not only to preserve the semantics of the original program but also to generate efficient and executable code for the target platform. [2]

A central challenge in compiler technology is: How can we trust compilers?. As discussed previously, compilers are inherently complex systems, particularly optimizing compilers, which perform intricate symbolic transformations. Despite rigorous testing practices, compilers can still introduce subtle bugs during these transformations. Such bugs are often extremely difficult to detect and diagnose [28], as they may lead to unexpected program crashes, incorrect behaviour, or silent miscomputations in the generated code, even when the original source code remains syntactically and semantically valid.

This raises serious concerns in the context of safety-critical or high-assurance software, where traditional validation through testing alone is insufficient. In such domains, testing must be complemented or in some cases replaced by formal methods, such as model checking, static analysis, and deductive program verification [28].

This is precisely where certified compilers become essential. A certified compiler is accompanied by a machine-checked formal proof that guarantees semantic preservation during the transformation from source to target code. It preserves the semantics of the source program during its transformation into target code. This approach offers strong assurances about the absence of certain classes of errors, such as compilation

bugs, compilation inaccuracies, or unsafe optimizations [29]. Unlike traditional compilers, whose correctness is typically established through empirical testing or informal reasoning, certified compilers provide mathematical guarantees of correctness, making them valuable in the development of high-assurance and safety-critical software.

### 3.1.1 CompCert

The development of a realistic and verified compiler began with CompCert. Here, verified denotes a compiler accompanied by a machine-checked proof that the generated code behaves exactly as prescribed by the semantics of the source program. Realistic refers to a compiler that can be effectively employed in the production of critical software systems [28].

CompCert adopts a multi-pass compilation architecture, where each pass translates an intermediate representation into a lower-level form, gradually transforming the high-level C source code into target assembly code. These intermediate languages are Clight, Cminor, RTL, LTL, and others which are formally defined within Coq [40]. CompCert's core is implemented in Gallina, the functional programming language of Coq, which is based on the Calculus of Inductive Constructions, a powerful higher-order logic and typed $\lambda$-calculus. This implementation enables formal reasoning and machine-checked proofs of correctness for each compilation phase [35].

Although CompCert is developed in Coq, it is not executed within the proof assistant. Instead, the verified Gallina code is extracted to OCaml, where it is combined with a small portion of handwritten OCaml code to produce an efficient and executable compiler [35].

Since the compiler must generate a large subset of the C language, the code needs to be efficient enough and compact enough to fit the requirements of critical embedded systems. This implies a multi-pass compiler that features good register allocation and some basic optimizations [28]. The pipeline for the compiler is presented below:

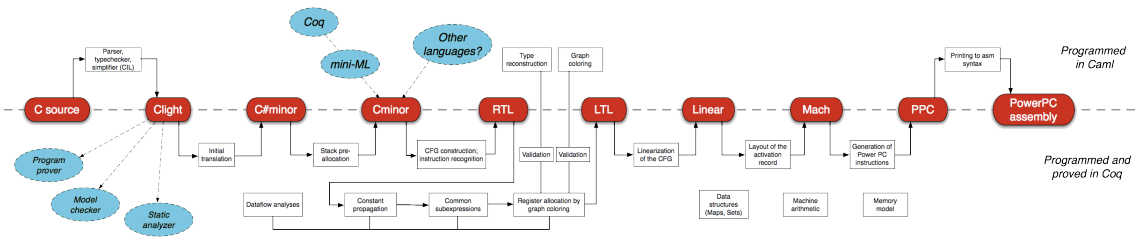


Figure 3.1: CompCert pipeline diagram [23, 1]

### 3.1.2 CakeML

The CakeML compiler stands out as one of the most realistic verified compilers for functional programming. Inspired by both CompCert and OCaml compilers, it supports a rich set of language features commonly found in unverified compilers for functional languages. The main goal of the CakeML compiler is to combine end-to-end formal

verification with practical performance, making it suitable for high-assurance applications without compromising efficiency [50]. CakeML is a formally verified functional programming language and compiler tool chain, specifically designed to target practical, off-the-shelf hardware while achieving a high degree of trustworthiness. It serves as a robust platform for software in domains where correctness is paramount. By focusing on functional programming and end-to-end verification, CakeML complements CompCert, which primarily target imperative languages and emphasize performance oriented optimizations. Together, these approaches address different but complementary aspects of verified software development [24].

The programming language used in CakeML is a subset of Standard ML, which is a functional programming language that fully embraces the expressiveness of mathematical functions. However, it was also shaped by practical programming needs, leading to the inclusion of imperative constructs and a robust exception handling system. The language supports modularity through an advanced system of parametric modules, designed to facilitate the structured development of large-scale software systems. Moreover, Standard ML is strongly and statically typed, and it was the first programming language to introduce a form of polymorphic type inference that combines strong type safety with considerable flexibility in programming style [34].

One of the most distinguishing features of Standard ML is its formal definition, which precisely specifies the language's static and dynamic semantics using structural operational semantics [34], which played a key role in laying the groundwork for end-to-end verified compilers [48].

In CakeML it is possible to find a verified type system, which is also an important achievement in the context of a certified compiler. It provides a sound and complete implementation of a system with type inference making use of the type soundness theorem and using HOL4 [46] proof assistant. These results ensure that any CakeML program accepted by the type inferencer is guaranteed to be well-typed and thus have well-defined semantics. This contributes to CakeML's overall goal of ensuring correctness by construction throughout the pipeline [49].

CakeML can also bootstrap its compiler, meaning it can compile itself, in order to be used inside and outside the logic. This bootstrapping is performed by generating verified machine code from a formally proven semantics of the compiler, effectively closing the trusted computing base gap. This process ensures that the final executable compiler has been produced through a chain of correctness preserving transformations, all verified in the HOL4 theorem prover [50, 24].

Complementing this, CakeML adopts a multi-stage compilation pipeline, Each stage of the pipeline transforms an intermediate representation of the program into a successively lower-level form, progressing from functional language down to executable machine code. The latest verified CakeML compiler and pipeline, as of writing this document, goes through 8 intermediate languages and targets machine code for 6 architectures:

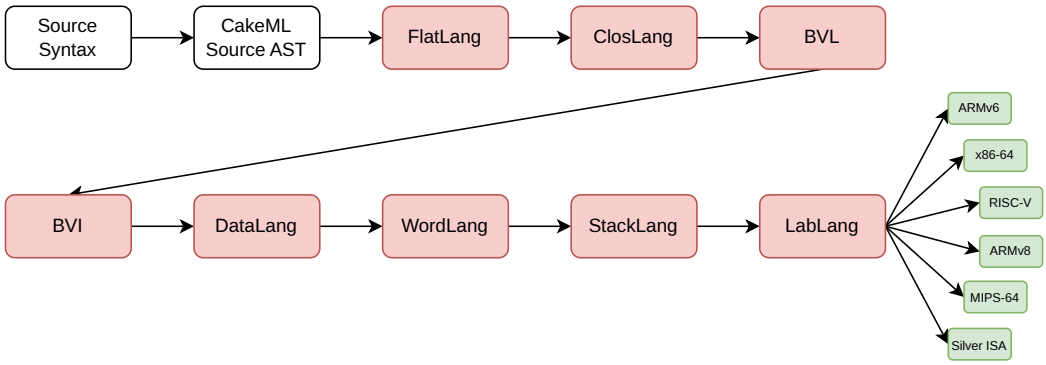


Figure 3.2: CakeML multi-stage pipeline

In the figure 3.2 the red boxes represent all the languages the compiler passes through until it reaches the encoding for each of the different types of machine code programs, represented as green.

### 3.1.3 PureCake

Another tool that compiles to CakeML code with a verified compiler is PureCake, which is a HOL4-verified compiler for a lazy, functional language with Haskell-like, indentation sensitive syntax. Since, the output code is CakeML code it can be passed through CakeML trusted backend, ensuring end-to-end verification [22].

Inspired by CompCert, it targets a Standard ML-style language and compiles down to CakeML through a verified pipeline. This design not only makes it possible to compile lazy functional programs in a sound and verified way, but also showcases how flexible CakeML's verified infrastructure can be when adapting to different programming language styles. By supporting features that are rarely seen in other verified compilers, PureCake expands the range of languages that can benefit from verified compilation, showing that it's feasible to bring formal guarantees even to non-strict, high-level functional languages [22, 21].

## 3.2 Pipeline

In this section we reintroduce the Cameleer pipeline found in figure 2.1, but now with some slight changes to accommodate the extraction mechanism from Why3:

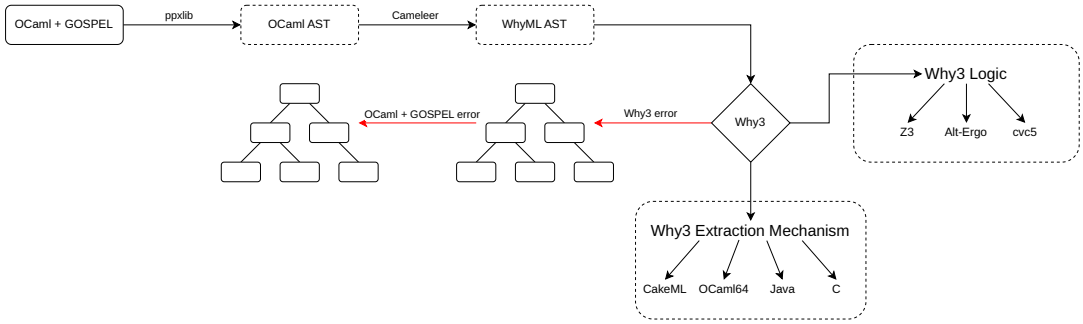


Figure 3.3: OCaml to WhyML pipeline with Cameleer

A brief explanation for the extended Cameleer pipeline, is that Cameleer translates OCaml programs with GOSPEL annotations to WhyML language internally, on one hand if an error is caught relating to OCaml or WhyML it will be handled accordingly, on the other hand if there is no syntax error the Why3 platform is initialized and the provers can be used to verify the proof goals.

The extraction mechanism introduced in the figure 3.3 is separate from the verification process. This means that a program may be extracted to another language without requiring the correctness of the code to be proven, the only guarantee is that the program does have any OCaml or GOSPEL compilation errors. This may lead to extraction of potentially incorrect OCaml programs, with the full responsibility of extracting after proving the code falling on the user's side.

The initial objective of the extraction mechanism was to build correct by construction libraries with the help of Why3. This lead to the creation of a mechanism that in itself does not jeopardize the behaviour of the original code when translating, this is due to its mathematical formalization [42]. One of the main advantages is the existence of a dedicated phase for the removal of ghost code and logical annotations present in the original WhyML code, in addition to a few conservative optimizations. The resulting code does not contain any logical elements which facilitates the translation to a new language, this means the only effort that needs to be done is similar to writing a printer from an intermediate representation of the language to be added [42]. Currently, four languages are allowed to be extracted from WhyML while using the Why3 platform, these are C, Java, OCaml (OCaml64) and CakeML [53, 42].

# 4

# Pipeline

This chapter presents the goal pipeline, the initial tool modifications already implemented and a set of manually derived examples that illustrate how OCaml + GOSPEL code can be correctly translated into semantically equivalent CakeML representations. These examples serve as a reference for the expected behaviour and structure of verified translations. The goal is to demonstrate the existing tools for translation into CakeML and identifying what the corresponding CakeML code should look like in order to compile successfully and preserve the intended behaviour.

## 4.1 Goal Pipeline

After analysing the verification pipeline in figure 3.3 it is possible to observe that some parts have already been implemented or require only minor adjustments to meet their objectives. By modifying the extraction process in Why3 to check if the proof has been discharged we can provide better correctness guarantees because the generated CakeML code will comply to the specification. Due to differences in syntax and features that have no direct equivalents in CakeML, we must also include some kind of error message for failures in extraction, for instance the lack of support for `while` and `for` loops.

The main goal of this work is to expand the currently available pipeline of translating code from OCaml with GOSPEL specifications into WhyML, where it can be verified using the various automated provers available in Why3. We want to achieve a more robust extraction mechanism with the ideas as previously discussed. Moreover, we ought to provide a new tool that translates a subset of compilable CakeML code with GOSPEL annotations into OCaml with GOSPEL equivalents that can later prove their correctness in Cameleer. Creating this translation tool for a certified compiler like CakeML is particularly valuable for research and comparative analysis. The figure below represents the goal pipeline:

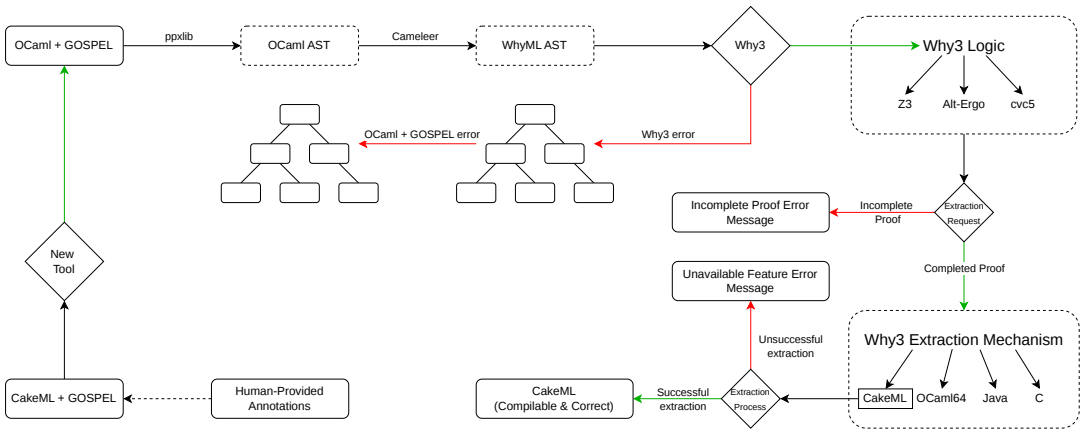


Figure 4.1: Goal pipeline

The pipeline in the previous figure 4.1 shows the two different approaches for deductive verification, extraction-based verification and translation-based verification. These two approaches for deductive verification can be put through the same case studies, in order to take conclusions from their trust in the pipeline, ease of automation and the expressiveness in the specifications.

## 4.2 Tool Modifications

As shown in Figure 4.1, which outlines the Cameleer pipeline, there is indirect support for CakeML through Cameleer, but the current extraction mechanism uses the OCaml64 option. A small modification to the file `/bin/cli.ml` of Cameleer's source code [1] [2], allows the extraction to CakeML. By changing the `execute_extract` function to invoke `why3 extract -D cakeml %s`, the tool can target CakeML as a backend. With this adjustment, running `cameleer program.ml --extract` translates the original OCaml plus GOSPEL code into code in the CakeML language. This enables extraction from OCaml directly into CakeML, providing the basis for a verified compilation pipeline.

The alternative to this process is to obtain the code translated WhyML from running Cameleer with the flag `--debug` and then apply the mechanism of extraction of Why3 directly. Not only is this process more complex, but also the debug flag produces code that can not be opened by the Why3 platform because it contains proof goals directly in the code. This also implies altering the code manually or to substantially modify the behaviour of the debug flag which is not intended.

[1] Original Cameleer code: https://github.com/ocaml-gospel/cameleer/blob/master/bin/cli.ml
[2] Modified Cameleer code: https://github.com/TiagoMeirim/cameleer/blob/master/bin/cli.ml

## 4.3 Case Studies

For all the case studies below mentioned, the OCaml + GOSPEL code was verified through Why3 automatic provers before applying the extraction command to translate into CakeML. This verification step is essential because the correctness guarantees of the final CakeML code depend entirely on the validity of the specifications and their successful discharge by the provers. Without ensuring that the WhyML intermediate code satisfies the given specifications, the generated CakeML code would no longer carry the desired formal guarantees, thereby breaking the intended end-to-end verification pipeline.

Since this step is intended only to demonstrate what already exists, the generated code may contain significant syntax errors and may not be compilable. These issues reflect limitations in the current extraction process and highlight the need for further refinement.

For presentation purposes we omitted parts of the original OCaml + GOSPEL code in some of the case studies. The complete versions can be found in the appendix A.

### 4.3.1 Tail Recursive Factorial

Initially an iterative version of the factorial function was also considered, however, since CakeML does not support imperative loop constructs such as `for` or `while` we had to look for alternative versions of this example. Instead, we opted to study the tail recursive variant to simulate iteration, found in section 2.5.1. The extracted code can be seen below:

*CakeML*

```
fun fact_aux n c t = let val n1 = n in
  let val c1 = c in
  let val t1 = t in
  if c1 <= n1 then (fact_aux n1 (c1 + (1)) (t1 * c1))  else (t1)

fun fact n = let val n1 = n in fact_aux n1 (1) (1)
```

The generated CakeML code failed to compile due to differences in `let` expression syntax between OCaml and CakeML. In CakeML, every `let` block must be explicitly closed with an `end` keyword, whereas in OCaml, this is not required. It should also be mentioned that the generated let-bindings generated are not necessary and only duplicate the variables in the arguments.

Initially we want to correct the let-binding termination as demonstrated in the following code:

*CakeML*

```
fun fact_aux n c t = let val n1 = n in
  let val c1 = c in
  let val t1 = t in
  if c1 <= n1 then (fact_aux n1 (c1 + (1)) (t1 * c1))  else (t1)
  end end end

fun fact n = let val n1 = n in fact_aux n1 (1) (1) end
```

However, our final goal is to eliminate as much unnecessary code as possible, in this case, the expendable let-bindings, while maintaining the desired behaviour:

```
fun fact_aux n c t =                                                    CakeML
  if c <= n then (fact_aux n (c + (1)) (t * c))  else (t)

fun fact n = fact_aux n (1) (1)
```

Both versions of this example are compilable and function correctly. This first example is simple and quite clearly demonstrates that our pipeline can function if small changes are applied to the extraction mechanism.

### 4.3.2 Exception recursive search

One possible use of exceptions, although it may seem natural at first, can be to stop iteration early in OCaml, for instance, in the context of linear search when the desired element is found, for more information regarding the implementation see section 2.3.4. Once again, since, for and while loops are not available in CakeML to simulate iteration we use recursion with a counter:

```
let rec search_aux a c n = (* ... *)                                GOSPEL + OCaml
(*@
  r = search_aux a c n
  requires 0 <= c <= Array.length a
  requires forall k. 0 <= k < c -> a.(k) <> n
  raises Not_found -> forall k. 0 <= k < Array.length a -> a.(k) <> n
  ensures 0 <= r < Array.length a
  ensures a.(r) = n
  variant Array.length a - c
*)
```

Inside the GOSPEL specifications, we must guarantee that index `c` is between values of 0 and the length of the array `a`, which in turn gives us the assurance that the array is never accessed for values out of its range expect for the case where `c` is equal to the length in which case the function terminates without accessing the array. To further strengthen the precondition above there is the need to state that all the values previously seen are not the desired value, otherwise the function should have terminated earlier.

Since the exception `Not_Found` was not handled this means that the function does not return a value in case it is raised, therefore only when a value is found it terminates successfully. In that case, we need to express two ensure clauses, one that states the resulting index is a value between 0 and the length of the array a minus one and that the element of the array in the index returned is effectively the value searching for. As the index is increasing in each iteration, at some point it becomes equal to the length of the array, thereby proving termination and the variant clause.

The core function, `linear_search`, delegates the search process to the auxiliary function that recursively traverses the array:

```
let linear_search a n = search_aux a 0 n                            GOSPEL + OCaml
(*@
```

```
  r = linear_search a n
  raises Not_found -> forall k. 0 <= k < Array.length a -> a.(k) <> n
  ensures 0 <= r < Array.length a
  ensures a.(r) = n
*)
```

Following the same train of thought, we must express the two possible outcomes. The first is when the `Not_found` exception is raised and the element was not found within the boundaries of the array. The second case is when the result is in fact an integer inside the boundaries and the value inside the array in the index provided is the same value being searched.

This makes the function verifiable by Why3 before attempting extraction to CakeML, ensuring that key correctness properties are formally validated. The extracted code is:

*CakeML*

```
fun search_aux a c n = let val a1 = a in
  let val c1 = c in
  let val n1 = n in
  let exception Break of (int) in
  ((if c1 = (a1.length) then (raise Not_found)
    else (if (get a1 c1) = n1 then (raise (Break n1))
          else (search_aux a1 (c1 + (1)) n1)))
  handle   Break i => i)

fun linear_search a n =
  let val a1 = a in let val n1 = n in search_aux a1 (0) n1
```

The generated CakeML code fails to compile due to several incompatibilities between OCaml and CakeML that must be addressed to ensure a successful and semantically correct extraction. One of the primary issues concerns exception handling, as CakeML requires all exceptions to be declared globally, outside the scope of functions, whereas the extracted code attempts to define the exception Break locally within a function using let-bound exception, a construct that is not valid in CakeML. This syntactic difference results in a compilation error and reflects a broader incompatibility in exception scoping between the two languages.

Additionally, array operations in CakeML differ substantially from those in OCaml. While OCaml allows for concise access through expressions like `a.(i)` while CakeML demands the usage of a function from the array library `Array.sub a i`. The generated code fails in this aspect by using invalid expressions such as `a1.length` and `get a1 c1`, neither are supported by the CakeML standard library nor array library. This divergence in the array library further contributes to the failure of the generated code to compile.

Moreover, the problems with let-bindings discussed earlier also contribute to another critical issue. These problems collectively illustrate the importance of a properly configured and semantically aware translation mechanism. The corrected code is presented below:

*CakeML*

```
exception Not_found
exception Break int
```

```
fun search_aux a c n =
  ((if c = (Array.length a) then (raise Not_found)
    else (if (Array.sub  a c) = n then (raise (Break n))
          else (search_aux a (c + (1)) n)))
  handle Break i => i)

fun linear_search a n = search_aux a (0) n
```

With this example we wanted to showcase how the extraction mechanism translates array definitions, access and other operations as well as handling exceptions.

### 4.3.3 Count Even For A List

Since operations for arrays have already been explored in the previous case study, we also want to explore if the same happens with operations for lists. We also utilize the mod operator from the standard library, when trying to count every even number inside a given list:

*GOSPEL + OCaml*

```
let count_even l =
  let l1 = List.filter (fun x -> x > 0 && x mod 2 = 0) l in
  let l2 = List.rev l1 in
  (List.length l2, l2)
(*@ (s, r) = count_even l
    ensures s = List.length r
    ensures forall x: int. List.mem x r -> x > 0 && mod x 2 = 0
    ensures forall x: int. List.mem x r -> List.mem x l
    ensures forall x: int. not List.mem x l -> not List.mem x r *)
```

The function `count_even` receives as the sole argument the list to be searched. The following code for the `l1` is to apply a filter for the original list `l`, where inside the filter we only want values that are positive and even. Theoretically, the operation is already done, but for research purposes we added more operations to see how the translation would undergo for different operations, those where the inversion of the list and the length of a list.

The specification passes through ensuring some crucial information. As the function is organized, it returns a tuple with both the corresponding length of the list created and the list created, so, we start by comparing the first element of the tuple, which should be the length of the reversed and filtered list, and therefore equal to the length of the second argument, the same list. Then there is a need to guarantee all the values inside the created list are effectively positive and even. Additionally, every element belonging inside the list created is also inside the original list and every value that is not inside the original list is not inside the created list. For simplicity, we do not guarantee that the number of occurrences for each number that belong to the resulting list is also the same as the original list. This condition would guarantee of this function however its proof would be complex to display here.

The extracted code can be seen below:

```
fun count_even l =                                                    CakeML
  let val l1 = l in
  let val l11 = filter (fn x => (x > (0)) andalso ((mod x (2)) = (0))) l1 in
  let val l2 = rev l11 in (List.length l2, l2)
```

The list operations are not correctly declared, it seems only the length operation is written with the correct syntax. The mod operator is used as if it is a function from the standard library, however in CakeML it is part of the `Int` library. The duplicate let-bindings were also generated for every argument.

Correcting the code, becomes the block below:

```
fun count_even l =                                                    CakeML
  let val l1 = List.filter (fn x => (x > (0)) andalso ((Int.mod x (2)) = (0))) l in
  let val l2 = List.rev l1 in (List.length l2, l2) end end
```

Using the correct syntax for calling every list operation and integer operation while deleting all the unnecessary let-bindings makes the code compilable and correct.

### 4.3.4 Depth-search tree

User defined data types, such as trees, are very important for real world examples in functional programming, since traversing them can easily be achieved recursively. For the tree data structure definition can be found in section 2.3.2. The recursive depth-first search algorithm for trees can be implemented as:

```
let rec depth_search t n =                                            GOSPEL + OCaml
  match t with
  | Leaf -> false
  | Node (l,x,r) -> x = n || depth_search l n || depth_search r n
(*@
  r = depth_search t n
  variant t
  ensures List.mem n (to_list t) <-> r
*)
```

The following recursive function `depth_search` shows how a tree is traversed through its depth firstly when trying to find a node that has the same value as the value given in the arguments.

Since the algorithm searches the subtree to the left and subtree to the right it is known that each of those trees is smaller than their parent. In the worst case, where the element is not found in the tree, we know that recursion terminates with the leaves, so the variant clause is the tree that was searched. One way to guarantee the result is correct is to transform the tree into a list and checking if the desired element is found in the resulting list with the `List.mem` operation.

To transform the tree into a list we formed an auxiliary logical function:

```
(*@ function rec to_list (t: 'a tree) : 'a list =                     GOSPEL + OCaml
```

```
  match t with
  | Leaf -> []
  | Node l x r -> x :: to_list l @ to_list r
*)
(*@
  variant t
*)
```

The recursive function `to_list` transforms a tree firstly from the own element of the root, following the left side and ending with the right side. Since, for this example, we are only concerned with the inclusion of an element in the list, the order in which it is constructed doesn't have effect on the outcome of the solution, our decision was arbitrary.

Similarly to the function `depth_search`, the termination can be proven by the tree itself.

The generated CakeML code was:

```
'a datatype tree = Leaf | Node of 'a tree * 'a * 'a tree
```

*CakeML*

```
fun depth_search t n = let val t1 = t in
  let val n1 = n in
  (case t1 of
    Leaf => false
  | Node l x r =>
    (x = n1) orelse ((depth_search l n1) orelse (depth_search r n1)))
```

The definition of the tree data type is incorrect for three reasons. Number one the ordering of the syntax for the left side declaration of a data type starts with the token `datatype` and only after the generic type `'a`. The second reason is the unnecessary token `of` that is not used when declaring a tuple in CakeML syntax. Thirdly, the tokens `*` are also unnecessary when declaring tuples. Finally, the let-bindings displayed the same errors as previous examples.

The corrected code below:

```
datatype 'a tree = Leaf | Node ('a tree) 'a ('a tree)
```

*CakeML*

```
fun depth_search t n =
  (case t of
    Leaf => False
  | Node l x r =>
    (x = n) orelse ((depth_search l n) orelse (depth_search r n)))
```

Correcting the order and removing the unnecessary tokens in data type definition, achieves a more robust extraction mechanism that produces compilable CakeML code.

### 4.3.5 Tree Comparison

With this example we only want to understand if the module system is working as intended in CakeML. The module system in CakeML is very limited since it does not support signatures and functors. In section 2.3.5 we present a simple tree module in

OCaml that contains the comparison between trees. Below it is shown the necessary GOSPEL annotations:

*GOSPEL + OCaml*

```
module Tree = struct
  type 'a tree = (* ... *)

  let rec cmp t1 t2 = (* ... *)
  (*@
  r = cmp t1 t2
  variant t1
  ensures r <-> t1 = t2
  *)
end
```

Logically, the result should be equal to comparing `t1` and `t2` and the variant clause could be either one of the trees since it only matters when the result is correct therefore the trees have the same structure, and in each recursive call we are considering fewer elements until, eventually, there are no more elements.

Generated code to CakeML was:

*CakeML*

```
'a datatype tree = Leaf | Node of 'a tree * 'a * 'a tree

fun cmp t1 t2 = let val t11 = t1 in
  let val t21 = t2 in
  (case (t11, t21) of
    (Leaf, Leaf) => true
  | (Leaf, _) => false
  | (_, Leaf) => false
  | (Node l1 x1 r1, Node l2 x2 r2) =>
    (cmp l1 l2) andalso ((x1 = x2) andalso (cmp r1 r2)))
```

Once more, the data types and let-bindings expressions failed to extract correctly, but more notably the module is not present. The corrected version is demonstrated below:

*CakeML*

```
structure Tree = struct
  datatype 'a tree = Leaf | Node ('a tree) 'a ('a tree)

  fun cmp t1 t2 =
  (case (t1, t2) of
    (Leaf, Leaf) => True
  | (Leaf, _) => False
  | (_, Leaf) => False
  | (Node l1 x1 r1, Node l2 x2 r2) =>
      (cmp l1 l2) andalso ((x1 = x2) andalso (cmp r1 r2)))
end
```

For the same problems, the same solutions were implemented, While modules are not explicitly shown in CakeML's documentation, by having a look at SML's module syntax, it was possible to reach the `structure Tree = struct ... end` construct, which was not presented in the translated code.

### 4.3.6 References

To demonstrate support for references, a program was implemented to perform Euclidean division, also known as division with remainder. This operation involves dividing one non-negative integer by another positive integer, resulting in two values: a positive integer quotient and a non-negative remainder that is strictly less than the absolute value of the divisor. This program is demonstrated in the section 2.3.4 and its specification is presented below:

```
let rec eudiv_aux (x: int) (y: int) (r: int ref) (q: int ref) = (* ... *)
(*@ res = eudiv_aux x y r q
    requires x >= 0
    requires y > 0
    requires 0 <= !r <= x
    requires x = y * !q + !r
    variant !r - y
    ensures x = y * !q + !r
    ensures 0 <= !r <= x
    ensures 0 <= !r < y *)
```

*GOSPEL + OCaml*

To specify this function, we used GOSPEL annotations to define pre-conditions and post-conditions that ensure the correctness of the Euclidean division. The pre-conditions enforce that the dividend must be a non-negative integer, the divisor must be strictly positive, and the remainder (held in a reference) must initially lie between 0 and the dividend. Additionally, we specify that the dividend is equal to the product of the divisor and quotient plus the remainder at each recursive call. For the post-conditions, we ensure that this equation is preserved after execution. Furthermore, the remainder must always remain non-negative and strictly less than the divisor, which aligns with the definition of Euclidean division. In every iteration, we subtract `y` from the remainder `r` so a natural way to guarantee termination is to use this exact expression as a variant condition.

```
let euclidean_div x y = (* ... *)
(*@ (q, r) = euclidean_div x y
    requires x >= 0
    requires y > 0
    ensures x = y * q + r
    ensures 0 <= r < y *)
```

*GOSPEL + OCaml*

The GOSPEL specifications require the dividend to be a non-negative integer and the divisor to be strictly positive. The function ensures that the dividend always equals the product of the divisor and the quotient plus the remainder. Additionally, it guarantees that the remainder remains non-negative and strictly less than the divisor, as defined by the Euclidean division properties.

The translated code to CakeML is represented below:

```
fun eudiv_aux x y r q = let val x1 = x in
  let val y1 = y in
  let val r1 = r in
```

*CakeML*

```
  let val q1 = q in
  if (!r1) >= y1
  then ((r1 := ((!r1) - y1); q1 := ((!q1) + (1)); eudiv_aux x1 y1 r1 q1))

fun euclidean_div x y =
  let val x1 = x in
  let val y1 = y in
  let val r = ref x1 in
  let val q = ref (0) in (eudiv_aux x1 y1 r q; (!q, !r))
```

The operations concerning references, creation, assignment and access, all seem to be well translated syntactically, the problem with let-bindings is recurring, however a new issue arises as the `else` branch with a single unit value was not generated for the function `eudiv_aux`.

The following code for CakeML is the corrected version:

*CakeML*

```
fun eudiv_aux x y r q =
  if (!r) >= y
  then ((r := ((!r) - y); q := ((!q) + (1)); eudiv_aux x y r q))
  else ()

fun euclidean_div x y =
  let val r = Ref x in
  let val q = Ref (0) in (eudiv_aux x y r q; (!q, !r))
    end end
```

Removing the unnecessary let-bindings and correctly handling of the `else` branch, which was absent, the functions became both syntactically correct and successfully compilable.

## 4.4 Analyses For The Case Studies

During our studies we found various issues, such as exception scoping, array operation semantics, and let-binding syntax, the extracted code remains invalid and cannot serve as a verified executable target. Therefore, improving the extraction pipeline from OCaml and GOSPEL to CakeML necessitates careful handling of these language specific constructs to ensure that the resulting code is not only syntactically correct but also preserves the behaviour of the original verified source.

# 5

# Extraction

The Extraction process, in the context of deductive program verification, refers to the process of deriving executable code from a verified specification by systematically eliminating all proof-oriented content that carries no computational meaning. Rather than re-verifying correctness in the target language, extraction relies on the guarantees already established during the verification phase, delegating to the extraction mechanism the responsibility of faithfully preserving the program's computational behaviour across the language boundary. In Why3, this process is formalized through a dedicated ghost code erasure phase followed by a language-specific driver, which together ensure that the logical constructs of a WhyML program spanning both program-level annotations such as pre-conditions, post-conditions, invariants, and ghost variables, and theory-level constructs such as predicates, lemmas, and axioms that support the proof obligations are cleanly removed before the remaining computational fragment is translated to the target language. It is precisely this translation step that introduces the constraints examined in this chapter, since the target language must be capable of representing the extracted fragment, the differences between WhyML and CakeML impose a well-defined boundary on the subset of WhyML constructs that can be viably used within the pipeline.

## 5.1 WhyML Subset

The integration of CakeML into the verification pipeline introduces a set of constraints that must be carefully accounted for. Unlike OCaml and WhyML, which belong to the ML language family but are designed with expressiveness and specification in mind, CakeML was developed primarily with compiler correctness as its central concern, making its language design reflect the requirements of a formally verified compilation chain rather than general-purpose functional programming. As a consequence, CakeML supports a more restricted set of language constructs than either OCaml or WhyML. This motivates the adoption of a specific subset of WhyML in the present work, one chosen to ensure that the programs expressed in the pipeline can be faithfully extracted to CakeML without introducing constructs that fall outside its supported fragment, thereby preserving

coherence across the full pipeline.

The subset was defined based on the limitations encountered during the extraction of WhyML code into CakeML. The first restriction concerns the character `o`, which in CakeML is strictly reserved for function composition and therefore cannot be used as an identifier for any variable or function name consisting solely of that character must be avoided within the subset. A further limitation arises with or-patterns since CakeML does not support the grouping of multiple patterns under a single branch, meaning that constructs of the form `| E | N l x r -> ...` are not accepted and must instead be decomposed into separate branches, inevitably introducing code duplication. Additionally, CakeML does not support imperative loop constructs such as `while` or `for`, requiring that all iterative behaviour be expressed through explicit recursion. Functors are likewise unsupported, and mutually recursive type definitions are not accepted by CakeML, both of which are consequently excluded from the subset.

It should be noted that the subset defined in this chapter is not automatically enforced by the pipeline. No static analysis or syntactic check is currently performed to verify that a given WhyML program operates in line with the subset defined before extraction. The user is therefore expected to respect this subset throughout the pipeline in order to ensure correctness across both the extraction and translation phases of the study.

## 5.2 Corrections to the CakeML Extraction Driver

The CakeML extraction driver was already partially implemented within the Why3 platform, however it contained a number of inaccuracies that produced syntactically incorrect CakeML output. The corrections addressed in this section are focused on the grammar of the generated code, ensuring that the extracted programs conform to the syntactic requirements of CakeML. As described in Section 4.2, extraction to CakeML is invoked via Cameleer using the `-extract` flag. The following subsections detail each individual change applied to the driver:

### 5.2.1 Capitalization Conventions

CakeML requires that certain types and literals be capitalized as part of its syntactic grammar. The previous version of the extraction driver did not respect this convention, producing lowercase identifiers where uppercase is grammatically required. Notable examples include the boolean literals `true` and `false`, which must be rendered as `True` and `False` respectively, as well as reference construction, where `ref` must appear as `Ref`.

### 5.2.2 Let Bindings Delimiters

A syntactic distinction between WhyML and CakeML concerns the delimitation of let bindings. In WhyML, let bindings do not require an explicit termination token, as the scope of the bound variable is determined by the structure of the expression. CakeML,

however, requires the `end` token to explicitly signal the closing of the binding scope. The correction applied to the extraction driver ensures that this token is consistently emitted at the appropriate position in the generated code.

### 5.2.3 Exception Handling

Exception handling introduces one of the more significant structural differences between WhyML and CakeML. While WhyML permits exception declarations within local scopes using let bindings, CakeML restricts exception declarations exclusively to the top level. To account for this, the extraction driver performs an initial pass over the function body to identify any exception declarations or raises, after which the declarations are hoisted to the top level and omitted from their original position in the generated code. Furthermore, the exceptions encountered during this work were not found in CakeML's standard library, which appears to offer limited support for the exceptions commonly assumed by WhyML programs, requiring them to be explicitly declared in the generated output.

A separate but related concern is the handling of the `absurd` construct. Since CakeML provides no equivalent, a pragmatic decision was made to map `absurd` to an exception, generating an `exception Absurd` declaration at the top level and a corresponding `raise Absurd` at each point of occurrence. This represents the simplest viable solution for preserving the intent of the construct when it comes to marking a branch as unreachable within CakeML's constraints.

## 5.3 Summary and Pipeline Implications

The majority of the corrections applied to the CakeML extraction driver are syntactic in nature, addressing grammatical discrepancies between WhyML and CakeML without affecting the verification guarantees established during the proof phase. Capitalization conventions, let binding delimiters, and exception handling are all corrections that determine whether the generated code is accepted by CakeML, but leave the computational semantics of the extracted program entirely unchanged.

The one notable exception is the handling of the `absurd` construct. In WhyML, `absurd` carries a precise logical meaning, it asserts that a given branch is unreachable, a claim that must be formally justified by a discharged proof obligation. Mapping this construct to `raise Absurd` in CakeML is ultimately not a purely syntactic transformation, it replaces a statically verified impossibility with a runtime exception. While raising an exception is not a perfect semantic equivalent of an unreachable branch, it represents a practical solution within CakeML's constraints, given that a correctly verified program should never reach that point at runtime.

# 6

# Translation

For the following chapter it is referred the creation from base the full translation of CakeML with the new GOSPEL annotations that were previously not present in the code. Unlike extraction, which derives executable code from a verified program by eliminating its logical content, translation operates in the opposite direction, taking an existing implementation together with its specifications and expressing both in WhyML so that verification can be carried out within the Why3 platform. The purpose of this process is not to replace extraction but to serve as a basis for comparison: by independently constructing a WhyML representation of the same program, it becomes possible to evaluate the two approaches against each other in terms of the verification effort required, the expressiveness of the resulting specifications, and the degree to which each method preserves the intent of the original code. The following section details the translation process and the decisions made when mapping CakeML and GOSPEL constructs into their WhyML counterparts.

## 6.1 Lexer

Before any syntactic or semantic analysis can take place, the raw source text must be broken down into meaningful units. This is the responsibility of the lexer, which scans the input sequentially and produces a stream of tokens that the parser can then process. Every raw source text that it is receiving regarding CakeML it will deconstruct it into tokens for the parser, on the other hand the raw text received inside the delimited GOSPEL specifications, in between the `(*@` and `*)`, will just return the raw text into parser so it can be deconstructed in another phase later.

## 6.2 Parser

The parser processes the token stream produced by the lexer and constructs an abstract syntax tree representing the input program. Each recognised is mapped to a corresponding AST node, producing a structured representation that the subsequent translation phase

can operate on directly. Operator precedence and associativity are resolved at this stage in accordance with CakeML's semantics, ensuring that the structure of expressions is faithfully preserved prior to translation into WhyML. Lastly, GOSPEL annotations are still preserved as raw strings in the abstract syntax tree, deferring their interpretation to a later stage of the translation process. The resulting AST serves as the central intermediate representation from which the translation into WhyML is driven.

## 6.3 Abstract Syntax Tree(AST)

The AST serves as the first structured internal representation of the input program, produced directly from the parsing phase. At this stage, the AST captures the full syntactic content of the CakeML source file, each represented as a typed node, that reflects the grammatical structure of the input. GOSPEL annotations, having been recognised by the lexer and passed through the parser as raw strings, are preserved at this stage without further interpretation, attached to their corresponding top-level positions in the tree.

## 6.4 GOSPEL Abstract Syntax Tree(GAST)

The GAST represents the final structured intermediate representation prior to translation into WhyML. It extends the AST by replacing raw GOSPEL annotation strings with fully parsed GOSPEL nodes, producing a unified representation in which CakeML program content and GOSPEL specifications coexist in a structured and typed form.

## 6.5 AST To GAST

The transformation from AST to GAST is responsible for promoting the raw GOSPEL annotation strings preserved in the AST into structured GOSPEL AST nodes, producing a unified representation in which each CakeML definition is paired with its corresponding specification. This phase is driven by the GOSPEL parser, which is invoked selectively depending on the nature of each annotation encountered.

A central distinction in this transformation is between *floating* and *inline* GOSPEL annotations. Floating annotations are top-level logical declarations that stand independently of any function definition, these include predicates, functions, axioms, and lemmas, which are parsed and mapped directly to their corresponding GAST nodes. Inline annotations, by contrast, are specifications attached to function definitions, such as pre-conditions and post-conditions, that are associated with the definition that follows or precedes them in the source file. To handle this, consecutive inline annotation strings are collected and merged before being passed to the GOSPEL parser as a single unified specification.

A further consideration arises from the fact that GOSPEL annotations may appear either before or after their associated function definition in the source file. The transformation

accounts for both orderings, collecting specification fragments from either side of the definition and combining them into a single coherent specification prior to parsing.

## 6.6 Printer

The final phase of the translation is the printer, where traversing the GAST and printing the correct WhyML code. While much of the translation is a straightforward structural mapping from CakeML constructs to their WhyML equivalents, several design decisions merit discussion from a verification standpoint.

Another decision to note is the automatic detection of recursive functions. Rather than requiring the user to explicitly mark recursive definitions, the printer analyses the function body to determine whether it contains a self-referential call, emitting the `rec` keyword accordingly. This analysis correctly accounts for shadowed identifiers, ensuring that a let binding or anonymous function argument sharing the function's name does not incorrectly trigger a recursive annotation.

Finally, the handling of `Absurd` in the printer is consistent with the decision taken in the extraction driver. Whereas extraction maps `absurd` to `raise Absurd`, the printer maps `raise Absurd` back to `absurd` in WhyML, restoring the original construct's semantics and ensuring that unreachable branches are correctly represented in the verification context.

# 7

# Comparative Evaluation

In order to create a fair comparison for both verification approaches the same use case will be used and analysed thoroughly for the extraction approach and translation approach. Before going into the results it is fundamental to talk about the impact of the subset in choosing the examples that can or not be used from the various verified examples from cameleer. The use case should, as always, respect the limitations implied from the subset chosen.

## 7.1 Verification Guarantees and Trust Boundaries

Before presenting the comparative results, it is necessary to characterize the trust boundaries of each approach and identify where verification guarantees can break down. Both approaches rely on an unverified component that constitutes the critical point of failure in the formal verification.

In the extraction approach, the code is verified within Why3 and the proof obligations are discharged before extraction is invoked 3.3. However, the extraction driver itself is not formally verified, meaning that if it fails to faithfully reproduce the WhyML semantics in the generated CakeML code, the verification guarantees established in Why3 are silently lost. In this scenario, even if the resulting CakeML code compiles successfully through the verified compiler, the compiler's correctness theorem offers no assurance that the verified properties are preserved, meaning the guarantee only extends from the CakeML source downwards, not from the WhyML source through the driver.

For the translation approach, the failure point arises earlier in the pipeline 4.1. If the translation tool incorrectly translates CakeML with GOSPEL specification into WhyML, the resulting WhyML program may not faithfully represent the original CakeML code, causing the verification step in Why3 to verify an inaccurate model. In this case, even if the proofs discharge successfully, they would be proving properties of the translated model rather than the original program. On the other hand, if the translation is faithful and the proofs discharge, the verified CakeML compiler provides a strong guarantee that the verified properties are preserved all the way to machine code.

The fundamental distinction between the two approaches therefore lies not in the verification mechanism itself, which is Why3 in both cases, but in the position of the unverified component and the consequences of its failure.

## 7.2 Comparative Case Studies

The case studies explored in these chapter were selected accordingly with the subset of the WhyML. For each example, the same program is subjected to both approaches independently, allowing a direct comparison of the verification effort required, the expressiveness of the resulting specifications, and the degree to which each method preserves the intent of the original code.

### 7.2.1 Red Black Tree

The Red-Black tree is a classical example of a self-balancing binary search tree in which each node is assigned a colour, either Red or Black, subject to two structural invariants: no two adjacent nodes may both be Red, and every path from a node to a null leaf must contain the same number of Black nodes[17]. These invariants together guarantee that the height of the tree remains bounded by $O(\log n)$, ensuring efficient worst-case performance for search, insertion, and deletion. Furthermore, if the tree as a whole satisfies these conditions, every subtree is also guaranteed to satisfy them, making the invariant compositional by nature.

#### 7.2.1.1 Extraction results

The verified code used in this case study is taken directly from the Why3 library of examples A.1.1, which provides a fully verified WhyML implementation of the Red-Black tree data structure, including all the necessary invariants and proof obligations. Since this implementation originates from the Why3 library, its correctness has already been established and all proof obligations are discharged, satisfying the prerequisite for extraction. The code was then subjected to the extraction mechanism described in Chapter 5, targeting CakeML as the output language.

The extraction process completed fully automatically, requiring no manual corrections to either the WhyML source or the generated CakeML output A.1.2. This result demonstrates that the corrected extraction driver is capable of handling a non-trivial verified data structure and producing compilable CakeML code without user intervention. This is a significant indicator of the robustness of the corrected pipeline for programs that operate within the defined WhyML subset.

#### 7.2.1.2 Translation results

For the translation approach, the CakeML code generated by the extraction process was annotated with GOSPEL specifications A.1.3, written manually to capture the intended

correctness properties of the Red-Black tree operations. This annotated CakeML code was then passed through the translation tool, which produced the corresponding WhyML code A.1.4.

The generated WhyML code was subsequently submitted to Why3, where all proof obligations were discharged automatically by the available provers, confirming that the translation process correctly preserved the verification intent of the manually written Gospel specifications.

## 7.3 Results Comparison

With the corrected Why3 extraction tool and the new translation tool, while always staying in line with the WhyML subset, it is evident from the results that both approaches work accordingly with what was expected in the expected pipeline. Both of the codes when passing through the verification tool proceeded correctly and with no errors.

# 8

# Conclusion

## 8.1 Future Work

## 8.2 Contributions

# Bibliography


[1] *"The structure of CompCert"*. Accessed: 2025-07-10. URL: https://www.absint.com/compcert/structure.htm (cit. on p. 17).

[2] A. V. Aho, R. Sethi, and J. D. Ullman. *Compilers: Principles, Techniques, and Tools*. Addison-Wesley series in computer science / World student series edition. Addison-Wesley, 1986. ISBN: 0-201-10088-6. URL: https://www.worldcat.org/oclc/12285707 (cit. on p. 16).

[3] H. Barbosa et al. "cvc5: A Versatile and Industrial-Strength SMT Solver". In: *Tools and Algorithms for the Construction and Analysis of Systems - 28th International Conference, TACAS 2022, Held as Part of the European Joint Conferences on Theory and Practice of Software, ETAPS 2022, Munich, Germany, April 2-7, 2022, Proceedings, Part I*. Ed. by D. Fisman and G. Rosu. Vol. 13243. Lecture Notes in Computer Science. Springer, 2022, pp. 415–442. DOI: 10.1007/978-3-030-99524-9_24. URL: https://doi.org/10.1007/978-3-030-99524-9%5C_24 (cit. on p. 12).

[4] C. Barrett and C. Tinelli. "Satisfiability Modulo Theories". In: *Handbook of Model Checking*. Ed. by E. M. Clarke et al. Cham: Springer International Publishing, 2018, pp. 305–343. ISBN: 978-3-319-10575-8. DOI: 10.1007/978-3-319-10575-8_11. URL: https://doi.org/10.1007/978-3-319-10575-8_11 (cit. on p. 1).

[5] F. Bobot et al. "Why3: Shepherd Your Herd of Provers". In: *Boogie 2011: First International Workshop on Intermediate Verification Languages*. https://hal.inria.fr/hal-00790310. Wrocław, Poland, 2011-08, pp. 53–64 (cit. on pp. 7, 12).

[6] A. Charguéraud et al. "GOSPEL - Providing OCaml with a Formal Specification Language". In: *Formal Methods - The Next 30 Years - Third World Congress, FM 2019, Porto, Portugal, October 7-11, 2019, Proceedings*. Ed. by M. H. ter Beek, A. McIver, and J. N. Oliveira. Vol. 11800. Lecture Notes in Computer Science. Springer, 2019, pp. 484–501. DOI: 10.1007/978-3-030-30942-8_29. URL: https://doi.org/10.1007/978-3-030-30942-8%5C_29 (cit. on pp. 1, 12).

[7] A. Church. "An Unsolvable Problem of Elementary Number Theory". In: *American Journal of Mathematics* 58.2 (1936-04), pp. 345–363. ISSN: 00029327. DOI: 10.2307/2371045. URL: http://dx.doi.org/10.2307/2371045 (cit. on p. 1).

[8] S. Conchon, M. Iguernelala, and A. Mebsout. "AltGr-Ergo, a Graphical User Interface for the SMT Solver Alt-Ergo". In: *Proceedings of the 12th Workshop on User Interfaces for Theorem Provers, UITP 2016, Coimbra, Portugal, 2nd July 2016*. Ed. by S. Autexier and P. Quaresma. Vol. 239. EPTCS. 2016, pp. 1–13. DOI: 10.4204/EPTCS.239.1. URL: https://doi.org/10.4204/EPTCS.239.1 (cit. on p. 12).

[9] S. A. Cook. "Soundness and Completeness of an Axiom System for Program Verification". In: *SIAM Journal on Computing* 7.1 (1978), pp. 70–90. DOI: 10.1137/0207005. eprint: https://doi.org/10.1137/0207005. URL: https://doi.org/10.1137/0207005 (cit. on p. 6).

[10] E. W. Dijkstra. *A Discipline of Programming*. Prentice-Hall, 1976. ISBN: 013215871X. URL: https://www.worldcat.org/oclc/01958445 (cit. on p. 7).

[11] J. Filliâtre. "Deductive software verification". In: *Int. J. Softw. Tools Technol. Transf.* 13.5 (2011), pp. 397–403. DOI: 10.1007/S10009-011-0211-0. URL: https://doi.org/10.1007/s10009-011-0211-0 (cit. on pp. 1, 13).

[12] J. Filliâtre and A. Paskevich. "Why3 - Where Programs Meet Provers". In: *Programming Languages and Systems - 22nd European Symposium on Programming, ESOP 2013, Held as Part of the European Joint Conferences on Theory and Practice of Software, ETAPS 2013, Rome, Italy, March 16-24, 2013. Proceedings*. Ed. by M. Felleisen and P. Gardner. Vol. 7792. Lecture Notes in Computer Science. Springer, 2013, pp. 125–128. DOI: 10.1007/978-3-642-37036-6_8. URL: https://doi.org/10.1007/978-3-642-37036-6%5C_8 (cit. on pp. 2, 11).

[13] J.-C. Filliâtre, M. Pereira, and S. M. de Sousa. "A Toolchain to Produce Correct-by-Construction OCaml Programs". In: *HAL Working Paper* (2018). Preprint, May 2018. URL: https://hal.science/hal-01783851v1/file/main.pdf (cit. on pp. 7, 8).

[14] R. W. Floyd. "Assigning Meanings to Programs". In: *Program Verification - Fundamental Issues in Computer Science*. Ed. by T. R. Colburn, J. H. Fetzer, and T. L. Rankin. Vol. 14. Studies in Cognitive Systems. Springer Netherlands, 1993, pp. 65–81. DOI: 10.1007/978-94-011-1793-7_4. URL: https://doi.org/10.1007/978-94-011-1793-7%5C_4 (cit. on p. 1).

[15] S. Goncharov et al. "Stateful Structural Operational Semantics". In: *7th International Conference on Formal Structures for Computation and Deduction, FSCD 2022, August 2-5, 2022, Haifa, Israel*. Ed. by A. P. Felty. Vol. 228. LIPIcs. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2022, 30:1–30:19. DOI: 10.4230/LIPICS.FSCD.2022.30. URL: https://doi.org/10.4230/LIPIcs.FSCD.2022.30 (cit. on p. 5).

[16] J. Gross et al. "Accelerating Verified-Compiler Development with a Verified Rewriting Engine". In: *13th International Conference on Interactive Theorem Proving, ITP 2022, August 7-10, 2022, Haifa, Israel*. Ed. by J. Andronick and L. de Moura. Vol. 237. LIPIcs. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2022, 17:1–17:18. DOI: 10.4230/LIPICS.ITP.2022.17. URL: https://doi.org/10.4230/LIPIcs.ITP.2022.17 (cit. on p. 1).

[17] L. J. Guibas and R. Sedgewick. "A Dichromatic Framework for Balanced Trees". In: *Proceedings of the 19th Annual Symposium on Foundations of Computer Science (FOCS)*. IEEE Computer Society, 1978, pp. 8–21. DOI: 10.1109/SFCS.1978.3 (cit. on p. 39).

[18] C. A. R. Hoare. "An Axiomatic Basis for Computer Programming". In: *Commun. ACM* 12.10 (1969), pp. 576–580. DOI: 10.1145/363235.363259. URL: https://doi.org/10.1145/363235.363259 (cit. on pp. 1, 6).

[19] B. Jacobs et al. "VeriFast: A Powerful, Sound, Predictable, Fast Verifier for C and Java". In: *NASA Formal Methods - Third International Symposium, NFM 2011, Pasadena, CA, USA, April 18-20, 2011. Proceedings*. Ed. by M. G. Bobaru et al. Vol. 6617. Lecture Notes in Computer Science. Springer, 2011, pp. 41–55. DOI: 10.1007/978-3-642-20398-5_4. URL: https://doi.org/10.1007/978-3-642-20398-5%5C_4 (cit. on p. 12).

[20] F. K. Jacobsen and J. Villadsen. "On Exams with the Isabelle Proof Assistant". In: *Electronic Proceedings in Theoretical Computer Science* 375 (2023-03), pp. 63–76. ISSN: 2075-2180. DOI: 10.4204/eptcs.375.6. URL: http://dx.doi.org/10.4204/EPTCS.375.6 (cit. on p. 11).

[21] H. Kanabar, K. Korban, and M. O. Myreen. "Verified Inlining and Specialisation for PureCake". In: *Programming Languages and Systems - 33rd European Symposium on Programming, ESOP 2024, Held as Part of the European Joint Conferences on Theory and Practice of Software, ETAPS 2024, Luxembourg City, Luxembourg, April 6-11, 2024, Proceedings, Part II*. Ed. by S. Weirich. Vol. 14577. Lecture Notes in Computer Science. Springer, 2024, pp. 275–301. DOI: 10.1007/978-3-031-57267-8_11. URL: https://doi.org/10.1007/978-3-031-57267-8%5C_11 (cit. on p. 19).

[22] H. Kanabar et al. "PureCake: A Verified Compiler for a Lazy Functional Language". In: *Proc. ACM Program. Lang.* 7.PLDI (2023), pp. 952–976. DOI: 10.1145/3591259. URL: https://doi.org/10.1145/3591259 (cit. on p. 19).

[23] D. Kästner et al. "CompCert: Practical Experience on Integrating and Qualifying a Formally Verified Optimizing Compiler". In: *ERTS2 2018 - 9th European Congress Embedded Real-Time Software and Systems*. 3AF, SEE, SIE. Toulouse, France, 2018-01, pp. 1–9. URL: https://inria.hal.science/hal-01643290 (cit. on p. 17).

[24] R. Kumar et al. "CakeML: A Verified Implementation of ML". In: *Principles of Programming Languages (POPL)*. ACM Press, 2014-01, pp. 179–191. DOI: 10.1145/2535838.2535841. URL: https://cakeml.org/popl14.pdf (cit. on pp. 1, 18).

[25] G. Leavens, A. Baker, and C. Ruby. "JML: a Java Modeling Language". In: (1998-10) (cit. on p. 12).

[26] K. R. M. Leino. "Dafny: An Automatic Program Verifier for Functional Correctness". In: *Logic for Programming, Artificial Intelligence, and Reasoning - 16th International Conference, LPAR-16, Dakar, Senegal, April 25-May 1, 2010, Revised Selected Papers*. Ed. by E. M. Clarke and A. Voronkov. Vol. 6355. Lecture Notes in Computer Science. Springer, 2010, pp. 348–370. DOI: 10.1007/978-3-642-17511-4_20. URL: https://doi.org/10.1007/978-3-642-17511-4%5C_20 (cit. on p. 7).

[27] X. Leroy. *"Formally verifying a compiler: what does it mean, exactly?"* Accessed: 2025-07-03. 2016. URL: https://xavierleroy.org/talks/ICALP2016.pdf (cit. on p. 1).

[28] X. Leroy. "A Formally Verified Compiler Back-end". In: *J. Autom. Reason.* 43.4 (2009), pp. 363–446. DOI: 10.1007/S10817-009-9155-4. URL: https://doi.org/10.1007/s10817-009-9155-4 (cit. on pp. 1, 16, 17).

[29] X. Leroy. "Formal verification of a realistic compiler". In: *Commun. ACM* 52.7 (2009), pp. 107–115. DOI: 10.1145/1538788.1538814. URL: https://doi.org/10.1145/1538788.1538814 (cit. on pp. 1, 2, 17).

[30] X. Leroy. *The ZINC experiment : an economical implementation of the ML language*. Tech. rep. RT-0117. INRIA, 1990-02, p. 100. URL: https://inria.hal.science/inria-00070049 (cit. on p. 7).

[31] X. Leroy and H. Grall. "Coinductive big-step operational semantics". In: *CoRR* abs/0808.0586 (2008). arXiv: 0808.0586. URL: http://arxiv.org/abs/0808.0586 (cit. on p. 5).

[32] A. Lööw et al. "Verified compilation on a verified processor". In: *Proceedings of the 40th ACM SIGPLAN Conference on Programming Language Design and Implementation, PLDI 2019, Phoenix, AZ, USA, June 22-26, 2019*. Ed. by K. S. McKinley and K. Fisher. ACM, 2019, pp. 1041–1053. DOI: 10.1145/3314221.3314622. URL: https://doi.org/10.1145/3314221.3314622 (cit. on p. 1).

[33] B. Meurer. "OCamlJIT 2.0 - Faster Objective Caml". In: *CoRR* abs/1011.1783 (2010). arXiv: 1011.1783. URL: http://arxiv.org/abs/1011.1783 (cit. on p. 8).

[34] R. Milner. *The Definition of Standard ML: Revised*. Mit Press. Penguin Random House LLC, 1997. ISBN: 9780262631815. URL: https://books.google.pt/books?id=e0PhKfbj-p8C (cit. on p. 18).

[35] D. Monniaux and S. Boulmé. "The Trusted Computing Base of the CompCert Verified Compiler". In: *Programming Languages and Systems - 31st European Symposium on Programming, ESOP 2022, Held as Part of the European Joint Conferences on Theory and Practice of Software, ETAPS 2022, Munich, Germany, April 2-7, 2022, Proceedings*. Ed. by I. Sergey. Vol. 13240. Lecture Notes in Computer Science. Springer, 2022,

pp. 204–233. DOI: 10.1007/978-3-030-99336-8_8. URL: https://doi.org/10.1007/978-3-030-99336-8%5C_8 (cit. on p. 17).

[36] F. L. Morris and C. B. Jones. "An Early Program Proof by Alan Turing". In: *IEEE Ann. Hist. Comput.* 6.2 (1984-04), pp. 139–143. ISSN: 1058-6180. DOI: 10.1109/MAHC.1984.10017. URL: https://doi.org/10.1109/MAHC.1984.10017 (cit. on p. 1).

[37] L. M. de Moura and N. S. Bjørner. "Z3: An Efficient SMT Solver". In: *Tools and Algorithms for the Construction and Analysis of Systems, 14th International Conference, TACAS 2008, Held as Part of the Joint European Conferences on Theory and Practice of Software, ETAPS 2008, Budapest, Hungary, March 29-April 6, 2008. Proceedings*. Ed. by C. R. Ramakrishnan and J. Rehof. Vol. 4963. Lecture Notes in Computer Science. Springer, 2008, pp. 337–340. DOI: 10.1007/978-3-540-78800-3_24. URL: https://doi.org/10.1007/978-3-540-78800-3%5C_24 (cit. on p. 12).

[38] P. Müller, M. Schwerhoff, and A. J. Summers. "Viper: A Verification Infrastructure for Permission-Based Reasoning". In: *Verification, Model Checking, and Abstract Interpretation - 17th International Conference, VMCAI 2016, St. Petersburg, FL, USA, January 17-19, 2016. Proceedings*. Ed. by B. Jobstmann and K. R. M. Leino. Vol. 9583. Lecture Notes in Computer Science. Springer, 2016, pp. 41–62. DOI: 10.1007/978-3-662-49122-5_2. URL: https://doi.org/10.1007/978-3-662-49122-5%5C_2 (cit. on p. 12).

[39] P. W. O'Hearn, J. C. Reynolds, and H. Yang. "Local Reasoning about Programs that Alter Data Structures". In: *Computer Science Logic, 15th International Workshop, CSL 2001. 10th Annual Conference of the EACSL, Paris, France, September 10-13, 2001, Proceedings*. Ed. by L. Fribourg. Vol. 2142. Lecture Notes in Computer Science. Springer, 2001, pp. 1–19. DOI: 10.1007/3-540-44802-0_1. URL: https://doi.org/10.1007/3-540-44802-0%5C_1 (cit. on p. 12).

[40] C. Paulin-Mohring. "Extracting omega's programs from proofs in the calculus of constructions". In: *Proceedings of the 16th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*. POPL '89. Austin, Texas, USA: Association for Computing Machinery, 1989, pp. 89–104. ISBN: 0897912942. DOI: 10.1145/75277.75285. URL: https://doi.org/10.1145/75277.75285 (cit. on pp. 2, 5, 11, 17).

[41] M. Pereira and A. Ravara. "Cameleer: A Deductive Verification Tool for OCaml". In: *Computer Aided Verification - 33rd International Conference, CAV 2021, Virtual Event, July 20-23, 2021, Proceedings, Part II*. Ed. by A. Silva and K. R. M. Leino. Vol. 12760. Lecture Notes in Computer Science. Springer, 2021, pp. 677–689. DOI: 10.1007/978-3-030-81688-9_31. URL: https://doi.org/10.1007/978-3-030-81688-9%5C_31 (cit. on pp. 2, 12, 13).

[42] M. J. P. Pereira. "Tools and Techniques for the Verification of Modular Stateful Code. (Outils et techniques pour la vérification de programmes impératives modulaires)".

PhD thesis. University of Paris-Saclay, France, 2018. URL: https://tel.archives-ouvertes.fr/tel-01980343 (cit. on p. 20).

[43] G. D. Plotkin. *A Structural Approach to Operational Semantics*. Tech. rep. DAIMI FN-19. University of Aarhus, 1981. URL: http://citeseer.ist.psu.edu/plotkin81structural.html (cit. on p. 5).

[44] A. Reynolds and C. Tinelli. "SyGuS Techniques in the Core of an SMT Solver". In: *Proceedings Sixth Workshop on Synthesis, SYNT@CAV 2017, Heidelberg, Germany, 22nd July 2017*. Ed. by D. Fisman and S. Jacobs. Vol. 260. EPTCS. 2017, pp. 81–96. DOI: 10.4204/EPTCS.260.8. URL: https://doi.org/10.4204/EPTCS.260.8 (cit. on p. 1).

[45] J. C. Reynolds. "Separation Logic: A Logic for Shared Mutable Data Structures". In: *17th IEEE Symposium on Logic in Computer Science (LICS 2002), 22-25 July 2002, Copenhagen, Denmark, Proceedings*. IEEE Computer Society, 2002, pp. 55–74. DOI: 10.1109/LICS.2002.1029817. URL: https://doi.org/10.1109/LICS.2002.1029817 (cit. on p. 12).

[46] K. Slind and M. Norrish. "A Brief Overview of HOL4". In: *Theorem Proving in Higher Order Logics, 21st International Conference, TPHOLs 2008, Montreal, Canada, August 18-21, 2008. Proceedings*. Ed. by O. A. Mohamed, C. A. Muñoz, and S. Tahar. Vol. 5170. Lecture Notes in Computer Science. Springer, 2008, pp. 28–32. DOI: 10.1007/978-3-540-71067-7_6. URL: https://doi.org/10.1007/978-3-540-71067-7%5C_6 (cit. on p. 18).

[47] *SPARK Reference Manual*. Accessed: 2025-07-11. 2022. URL: https://docs.adacore.com/spark2014-docs/html/lrm/ (cit. on p. 12).

[48] D. Syme. "Reasoning with the Formal Definition of Standard ML in HOL". In: *Higher Order Logic Theorem Proving and its Applications, 6th International Workshop, HUG '93, Vancouver, BC, Canada, August 11-13, 1993, Proceedings*. Ed. by J. J. Joyce and C. H. Seger. Vol. 780. Lecture Notes in Computer Science. Springer, 1993, pp. 43–60. DOI: 10.1007/3-540-57826-9_124. URL: https://doi.org/10.1007/3-540-57826-9%5C_124 (cit. on p. 18).

[49] Y. K. Tan, S. Owens, and R. Kumar. "A verified type system for CakeML". In: *Proceedings of the 27th Symposium on the Implementation and Application of Functional Programming Languages, IFL '15, Koblenz, Germany, September 14-16, 2015*. Ed. by R. Lämmel. ACM, 2015, 7:1–7:12. DOI: 10.1145/2897336.2897344. URL: https://doi.org/10.1145/2897336.2897344 (cit. on p. 18).

[50] Y. K. Tan et al. "The verified CakeML compiler backend". In: *J. Funct. Program.* 29 (2019), e2. DOI: 10.1017/S0956796818000229. URL: https://doi.org/10.1017/S0956796818000229 (cit. on p. 18).

[51] A. M. Turing. "On computable numbers, with an application to the Entscheidungsproblem". In: *Proc. London Math. Soc.* s2-42.1 (1937), pp. 230–265. DOI: 10.1112/PLMS/S2-42.1.230. URL: https://doi.org/10.1112/plms/s2-42.1.230 (cit. on p. 1).

[52] A. M. Turing. "Systems of Logic Based on Ordinals". PhD thesis. Princeton University, NJ, USA, 1938. DOI: 10.1112/PLMS/S2-45.1.161. URL: https://doi.org/10.1112/plms/s2-45.1.161 (cit. on p. 1).

[53] *Why Extraction Command*. Accessed: 2025-07-10. URL: www.why3.org/doc/manpages.html#why3:tool-extract (cit. on p. 20).

[54] R. Zach. "Chapter 71 - Kurt Gödel, paper on the incompleteness theorems (1931)". In: *Landmark Writings in Western Mathematics 1640-1940*. Ed. by I. Grattan-Guinness et al. Amsterdam: Elsevier Science, 2005, pp. 917–925. ISBN: 978-0-444-50871-3. DOI: https://doi.org/10.1016/B978-044450871-3/50152-2. URL: https://www.sciencedirect.com/science/article/pii/B9780444508713501522 (cit. on p. 1).

# A

# Complete Case Studies

## Tail Recursive Factorial

### OCaml + GOSPEL

*GOSPEL + OCaml*

```
(*@
  function rec factorial (n:int) :int =
    if n=0 then 1 else n * factorial (n-1)
*)
(*@
  requires n >= 0
  variant n
*)

let rec fact_aux n c t =
  if c <= n then fact_aux n (c+1) (t*c) else t
(*@
  r = fact_aux n c t
  requires n >= 0
  requires 0 < c <= n+1
  requires t = factorial (c-1)
  ensures r = factorial n
  variant n+1-c
*)

let fact n = fact_aux n 1 1
(*@
  r = fact n
  requires n >= 0
  ensures r = factorial n
*)
```

### Generated CakeML

CakeML

```
fun fact_aux n c t = let val n1 = n in
  let val c1 = c in
  let val t1 = t in
  if c1 <= n1 then (fact_aux n1 (c1 + (1)) (t1 * c1))  else (t1)

fun fact n = let val n1 = n in fact_aux n1 (1) (1)
```

### Corrected CakeML 1st Version

CakeML

```
fun fact_aux n c t = let val n1 = n in
  let val c1 = c in
  let val t1 = t in
  if c1 <= n1 then (fact_aux n1 (c1 + (1)) (t1 * c1))  else (t1)
  end end end

fun fact n = let val n1 = n in fact_aux n1 (1) (1) end
```

### Corrected CakeML 2nd Version

CakeML

```
fun fact_aux n c t =
  if c <= n then (fact_aux n (c + (1)) (t * c))  else (t)

fun fact n = fact_aux n (1) (1)
```

# Exception recursive search

## OCaml + GOSPEL

*GOSPEL + OCaml*

```
let rec search_aux a c n =
  let exception Break of int in try
    if c = Array.length a then raise Not_found
    else if a.(c) = n then raise (Break c)
    else search_aux a (c+1) n
  with Break i -> i
(*@
  r = search_aux a c n
  requires 0 <= c <= Array.length a
  requires forall k. 0 <= k < c -> a.(k) <> n
  raises Not_found -> forall k. 0 <= k < Array.length a -> a.(k) <> n
  ensures 0 <= r < Array.length a
  ensures a.(r) = n
  variant Array.length a - c
*)

let linear_search a n = search_aux a 0 n
(*@
  r = linear_search a n
  raises Not_found -> forall k. 0 <= k < Array.length a -> a.(k) <> n
  ensures 0 <= r < Array.length a
  ensures a.(r) = n
*)
```

## Generated CakeML

*CakeML*

```
fun search_aux a c n = let val a1 = a in
  let val c1 = c in
  let val n1 = n in
  let exception Break of (int) in
  ((if c1 = (a1.length) then (raise Not_found)
    else (if (get a1 c1) = n1 then (raise (Break n1))
          else (search_aux a1 (c1 + (1)) n1)))
  handle   Break i => i)

fun linear_search a n =
  let val a1 = a in let val n1 = n in search_aux a1 (0) n1
```

## Corrected CakeML

*CakeML*

```
exception Not_found
exception Break int

fun search_aux a c n =
  ((if c = (Array.length a) then (raise Not_found)
    else (if (Array.sub a c) = n then (raise (Break n))
          else (search_aux a (c + (1)) n)))
  handle Break i => i)

fun linear_search a n = search_aux a (0) n
```

# Count Even For A List

## OCaml + GOSPEL

*GOSPEL + OCaml*

```
let count_even l =
  let l1 = List.filter (fun x -> x > 0 && x mod 2 = 0) l in
  let l2 = List.rev l1 in
  (List.length l2, l2)
(*@ (s, r) = count_even l
    ensures s = List.length r
    ensures forall x: int. List.mem x r -> x > 0 && mod x 2 = 0
    ensures forall x: int. List.mem x r -> List.mem x l
    ensures forall x: int. not List.mem x l -> not List.mem x r *)
```

## Generated CakeML

*CakeML*

```
fun count_even l =
  let val l1 = l in
  let val l11 = filter (fn x => (x > (0)) andalso ((mod x (2)) = (0))) l1 in
  let val l2 = rev l11 in (List.length l2, l2)
```

## Corrected CakeML

*CakeML*

```
fun count_even l =
  let val l1 = List.filter (fn x => (x > (0)) andalso ((Int.mod x (2)) = (0))) l in
  let val l2 = List.rev l1 in (List.length l2, l2) end end
```

# Depth-search tree

## OCaml + GOSPEL

*GOSPEL + OCaml*

```
(*@ function rec to_list (t: 'a tree) : 'a list =
  match t with
  | Leaf -> []
  | Node l x r -> x :: to_list l @ to_list r
*)
(*@
  variant t
*)

let rec depth_search t n =
  match t with
  | Leaf -> false
  | Node (l,x,r) -> x = n || depth_search l n || depth_search r n
(*@
  r = depth_search t n
  variant t
  ensures List.mem n (to_list t) <-> r
*)
```

## Generated CakeML

*CakeML*

```
'a datatype tree = Leaf | Node of 'a tree * 'a * 'a tree

fun depth_search t n = let val t1 = t in
  let val n1 = n in
  (case t1 of
    Leaf => false
  | Node l x r =>
    (x = n1) orelse ((depth_search l n1) orelse (depth_search r n1)))
```

## Corrected CakeML

*CakeML*

```
datatype 'a tree = Leaf | Node ('a tree) 'a ('a tree)

fun depth_search t n =
  (case t of
    Leaf => False
  | Node l x r =>
    (x = n) orelse ((depth_search l n) orelse (depth_search r n)))
```

# Tree Comparison

## OCaml + GOSPEL

*GOSPEL + OCaml*

```
module Tree = struct
  type 'a tree =
    | Leaf
    | Node of 'a tree * 'a * 'a tree

  let rec cmp t1 t2 =
    match t1, t2 with
    | Leaf, Leaf -> true
    | Leaf, _ -> false
    | _, Leaf -> false
    | Node (l1,x1,r1), Node (l2,x2,r2) -> cmp l1 l2 && x1 = x2 && cmp r1 r2
  (*@
  r = cmp t1 t2
  variant t1
  ensures r <-> t1 = t2
  *)
end
```

## Generated CakeML

*CakeML*

```
'a datatype tree = Leaf | Node of 'a tree * 'a * 'a tree

fun cmp t1 t2 = let val t11 = t1 in
  let val t21 = t2 in
  (case (t11, t21) of
    (Leaf, Leaf) => true
  | (Leaf, _) => false
  | (_, Leaf) => false
  | (Node l1 x1 r1, Node l2 x2 r2) =>
    (cmp l1 l2) andalso ((x1 = x2) andalso (cmp r1 r2)))
```

## Corrected CakeML

*CakeML*

```
structure Tree = struct
  datatype 'a tree = Leaf | Node ('a tree) 'a ('a tree)

  fun cmp t1 t2 =
  (case (t1, t2) of
    (Leaf, Leaf) => True
  | (Leaf, _) => False
  | (_, Leaf) => False
  | (Node l1 x1 r1, Node l2 x2 r2) =>
      (cmp l1 l2) andalso ((x1 = x2) andalso (cmp r1 r2)))
end
```

# References

## OCaml + GOSPEL

*GOSPEL + OCaml*

```
let rec eudiv_aux (x: int) (y: int) (r: int ref) (q: int ref) =
  if !r >= y then (r := !r - y; q := !q + 1; eudiv_aux x y r q)
  else ()
(*@ res = eudiv_aux x y r q
    requires x >= 0
    requires y > 0
    requires 0 <= !r <= x
    requires x = y * !q + !r
    variant !r - y
    ensures x = y * !q + !r
    ensures 0 <= !r <= x
    ensures 0 <= !r < y *)

let euclidean_div x y =
    let r = ref x and q = ref 0 in
  eudiv_aux x y r q;
  (!q, !r)
(*@ (q, r) = euclidean_div x y
    requires x >= 0
    requires y > 0
    ensures x = y * q + r
    ensures 0 <= r < y *)
```

## Generated CakeML

*CakeML*

```
fun eudiv_aux x y r q = let val x1 = x in
  let val y1 = y in
  let val r1 = r in
  let val q1 = q in
  if (!r1) >= y1
  then ((r1 := ((!r1) - y1); q1 := ((!q1) + (1)); eudiv_aux x1 y1 r1 q1))

fun euclidean_div x y =
  let val x1 = x in
  let val y1 = y in
  let val r = ref x1 in
  let val q = ref (0) in (eudiv_aux x1 y1 r q; (!q, !r))
```

### Corrected CakeML

*CakeML*

```
fun eudiv_aux x y r q =
  if (!r) >= y
  then ((r := ((!r) - y); q := ((!q) + (1)); eudiv_aux x y r q))
  else ()

fun euclidean_div x y =
  let val r = Ref x in
  let val q = Ref (0) in (eudiv_aux x y r q; (!q, !r))
    end end
```

## A.1 Red Black Trees

### A.1.1 Verified WhyML Code

*WhyML*

```
module RedBlackTree

  (* Red-black trees (data type) *)

  type key = int
  type value = int

  type color = Red | Black

  type tree =
    | Leaf
    | Node color tree key value tree

  (* occurrence of a key/value pair in a tree *)

  predicate memt (t : tree) (k : key) (v : value) =
    match t with
    | Leaf -> false
    | Node _ l k' v' r -> (k = k' /\ v = v') \/ memt l k v \/ memt r k v
    end

  lemma memt_color :
    forall l r : tree, k k' : key, v v' : value, c c' : color.
    memt (Node c l k v r) k' v' -> memt (Node c' l k v r) k' v'

  (* binary search tree *)

  use int.Int

  predicate lt_tree (x : key) (t : tree) =
    forall k : key. forall v : value. memt t k v -> k < x

  predicate gt_tree (x : key) (t : tree) =
    forall k : key. forall v : value. memt t k v -> x < k
```

```
lemma lt_leaf: forall x: key. lt_tree x Leaf

lemma gt_leaf: forall x: key. gt_tree x Leaf

lemma lt_tree_node:
  forall x y: key, v: value, l r: tree, c: color.
  lt_tree x l -> lt_tree x r -> y < x -> lt_tree x (Node c l y v r)

lemma gt_tree_node:
  forall x y: key, v: value, l r: tree, c: color.
  gt_tree x l -> gt_tree x r -> x < y -> gt_tree x (Node c l y v r)

lemma lt_node_lt:
  forall x y: key, v: value, l r: tree, c: color.
  lt_tree x (Node c l y v r) -> y < x

lemma gt_node_gt:
  forall x y: key, v: value, l r: tree, c: color.
  gt_tree x (Node c l y v r) -> x < y

lemma lt_left:
  forall x y: key, v: value, l r: tree, c : color.
  lt_tree x (Node c l y v r) -> lt_tree x l

lemma lt_right:
  forall x y: key, v: value,  l r: tree, c: color.
  lt_tree x (Node c l y v r) -> lt_tree x r

lemma gt_left:
  forall x y: key, v: value, l r: tree, c: color.
  gt_tree x (Node c l y v r) -> gt_tree x l

lemma gt_right:
  forall x y: key, v: value, l r: tree, c: color.
  gt_tree x (Node c l y v r) -> gt_tree x r

lemma lt_tree_not_in:
  forall x: key, t: tree. lt_tree x t -> forall v: value. not (memt t x v)

lemma lt_tree_trans:
  forall x y: key. x < y -> forall t: tree. lt_tree x t -> lt_tree y t

lemma gt_tree_not_in :
  forall x: key, t: tree. gt_tree x t -> forall v: value. not (memt t x v)

lemma gt_tree_trans :
  forall x y: key. y < x -> forall t: tree. gt_tree x t -> gt_tree y t

predicate bst (t : tree) =
```

```
  match t with
  | Leaf -> true
  | Node _ l k _ r -> bst l /\ bst r /\ lt_tree k l /\ gt_tree k r
  end

lemma bst_Leaf : bst Leaf

lemma bst_left:
  forall k: key, v: value, l r: tree, c: color. bst (Node c l k v r) -> bst l

lemma bst_right:
  forall k: key, v: value, l r: tree, c: color. bst (Node c l k v r) -> bst r

lemma bst_color:
  forall c c': color, k: key, v: value, l r: tree.
  bst (Node c l k v r) -> bst (Node c' l k v r)

lemma rotate_left:
  forall kx ky: key, vx vy: value, a b c: tree, c1 c2 c3 c4: color.
  bst (Node c1 a kx vx (Node c2 b ky vy c)) ->
  bst (Node c3 (Node c4 a kx vx b) ky vy c)

lemma rotate_right:
  forall kx ky: key, vx vy: value, a b c: tree, c1 c2 c3 c4: color.
  bst (Node c3 (Node c4 a kx vx b) ky vy c) ->
  bst (Node c1 a kx vx (Node c2 b ky vy c))

predicate is_not_red (t : tree) =
  match t with
  | Node Red _ _ _ _ -> false
  | Leaf | Node Black _ _ _ _ -> true
  end

predicate rbtree (n : int) (t : tree) =
  match t with
  | Leaf ->
      n = 0
  | Node Red   l _ _ r ->
      rbtree n l /\ rbtree n r /\ is_not_red l /\ is_not_red r
  | Node Black l _ _ r ->
      rbtree (n-1) l /\ rbtree (n-1) r
  end

lemma rbtree_Leaf:
  rbtree 0 Leaf

lemma rbtree_Node1:
  forall k:key, v:value. rbtree 0 (Node Red Leaf k v Leaf)

lemma rbtree_left:
```

```
  forall x: key, v: value, l r: tree, c: color.
  (exists n: int. rbtree n (Node c l x v r)) -> exists n: int. rbtree n l

lemma rbtree_right:
  forall x: key, v: value, l r: tree, c: color.
  (exists n: int. rbtree n (Node c l x v r)) -> exists n: int. rbtree n r

exception Not_found

let rec find (t : tree) (k : key) : value
  requires { bst t }
  variant { t }
  ensures { memt t k result }
  raises { Not_found -> forall v : value. not (memt t k v) }
= match t with
  | Leaf -> raise Not_found
  | Node _ l k' v r ->
      if k = k' then v
      else if k < k' then find l k
      else (* k > k' *) find r k
  end

predicate almost_rbtree (n : int) (t : tree) =
  match t with
  | Leaf ->
      n = 0
  | Node Red   l _ _ r ->
      rbtree n l /\ rbtree n r
  | Node Black l _ _ r ->
      rbtree (n-1) l /\ rbtree (n-1) r
  end

lemma rbtree_almost_rbtree:
  forall n: int, t: tree. rbtree n t -> almost_rbtree n t

lemma rbtree_almost_rbtree_ex:
  forall s: tree.
  (exists n: int. rbtree n s) -> exists n: int. almost_rbtree n s

lemma almost_rbtree_rbtree_black:
  forall x: key, v: value, l r: tree, n: int.
  almost_rbtree n (Node Black l x v r) -> rbtree n (Node Black l x v r)

let lbalance (l : tree) (k : key) (v : value) (r : tree)
  requires { lt_tree k l /\ gt_tree k r /\ bst l /\ bst r }
  ensures { bst result /\
    (forall n : int. almost_rbtree n l -> rbtree n r -> rbtree (n+1) result)
    /\
    forall k':key, v':value.
      memt result k' v' <->
```

```
      if k' = k then v' = v else (memt l k' v' \/ memt r k' v') }
= match l with
  | Node Red (Node Red a kx vx b) ky vy c
  | Node Red a kx vx (Node Red b ky vy c) ->
      Node Red (Node Black a kx vx b) ky vy (Node Black c k v r)
  | _ ->
      Node Black l k v r
  end

let rbalance (l : tree) (k : key) (v : value) (r : tree)
  requires { lt_tree k l /\ gt_tree k r /\ bst l /\ bst r }
  ensures { bst result /\
    (forall n : int. almost_rbtree n r -> rbtree n l -> rbtree (n+1) result)
    /\
    forall k':key, v':value.
      memt result k' v' <->
      if k' = k then v' = v else (memt l k' v' \/ memt r k' v') }
= match r with
  | Node Red (Node Red b ky vy c) kz vz d
  | Node Red b ky vy (Node Red c kz vz d) ->
      Node Red (Node Black l k v b) ky vy (Node Black c kz vz d)
  | _ ->
      Node Black l k v r
  end

(* `insert x s` inserts `x` in tree `s`, resulting in a possible top red-red
   conflict when `s` is red. *)

let rec insert (t : tree) (k : key) (v : value) : tree
  requires { bst t /\ exists n: int. rbtree n t }
  variant { t }
  ensures { bst result /\
    (forall n : int. rbtree n t -> almost_rbtree n result /\
       (is_not_red t -> rbtree n result)) /\
    memt result k v /\
    forall k':key, v':value.
    memt result k' v' <-> if k' = k then v' = v else memt t k' v' }
= match t with
  | Leaf ->
      Node Red Leaf k v Leaf
  | Node Red l k' v' r ->
      if k < k' then Node Red (insert l k v) k' v' r
      else if k' < k then Node Red l k' v' (insert r k v)
      else (* k = k' *) Node Red l k' v r
  | Node Black l k' v' r ->
      if k < k' then lbalance (insert l k v) k' v' r
      else if k' < k then rbalance l k' v' (insert r k v)
      else (* k = k' *) Node Black l k' v r
  end
```

```
  (* finally ‘add x s‘ calls ‘insert‘ and recolors the root as black *)

  let add (t : tree) (k : key) (v : value) : tree
    requires { bst t /\ exists n:int. rbtree n t }
    ensures { bst result /\ (exists n:int. rbtree n result) /\
      memt result k v /\
      forall k':key, v':value.
      memt result k' v' <-> if k' = k then v' = v else memt t k' v' }
  = match insert t k v with
    | Node _ l k' v' r -> Node Black l k' v' r
    | Leaf -> absurd
    end

end
```

### A.1.2 Generated CakeML Code From Extraction

*CakeML*

```
type key = int

type value = int

datatype color = Red | Black

datatype tree = Leaf | Node color tree (int) (int) tree

exception Not_found



fun find t k =
  (case t of
    Leaf => raise Not_found
  | (Node _ l kqt v r) =>
    if k = kqt then (v)  else (if k < kqt then (find l k)  else (find r k)))

fun lbalance l k v r =
  (case l of
    (Node Red (Node Red a kx vx b) ky vy c) | (Node Red a kx vx (Node Red b
    ky vy c)) =>
    Node (Red) (Node (Black) a kx vx b) ky vy (Node (Black) c k v r)
  | _ => Node (Black) l k v r)

fun rbalance l k v r =
  (case r of
    (Node Red (Node Red b ky vy c) kz vz d) | (Node Red b ky vy (Node Red c
    kz vz d)) =>
    Node (Red) (Node (Black) l k v b) ky vy (Node (Black) c kz vz d)
  | _ => Node (Black) l k v r)

fun insert t k v =
```

```
  (case t of
    Leaf => Node (Red) (Leaf) k v (Leaf)
  | (Node Red l kqt vqt r) =>
    if k < kqt then (Node (Red) (insert l k v) kqt vqt r)
    else (if kqt < k then (Node (Red) l kqt vqt (insert r k v))
          else (Node (Red) l kqt v r))
  | (Node Black l kqt vqt r) =>
    if k < kqt then (lbalance (insert l k v) kqt vqt r)
    else (if kqt < k then (rbalance l kqt vqt (insert r k v))
          else (Node (Black) l kqt v r)))

exception Absurd

fun add t k v =
  (case insert t k v of
    (Node _ l kqt vqt r) => Node (Black) l kqt vqt r
  | Leaf => raise Absurd)
```

### A.1.3 Generated CakeML Code + GOSPEL Annotations

*CakeML*

```
type key = int
type value = int
datatype color = Red | Black
datatype tree = Leaf | Node color tree (int) (int) tree

(*@ predicate memt (t: tree) (k: key) (v: value) =
    match t with
    | Leaf -> false
    | Node _ l k' v' r -> (k = k' && v = v') || memt l k v || memt r k v *)

(*@ lemma memt_color :
    forall l r : tree, k k' : key, v v' : value, c c' : color.
    memt (Node c l k v r) k' v' -> memt (Node c' l k v r) k' v' *)

(*@ predicate lt_tree (x : key) (t : tree) =
    forall k : key. forall v : value. memt t k v -> k < x *)

(*@ predicate gt_tree (x : key) (t : tree) =
    forall k : key. forall v : value. memt t k v -> x < k *)

(*@ lemma lt_leaf: forall x: key. lt_tree x Leaf *)

(*@ lemma gt_leaf: forall x: key. gt_tree x Leaf *)

(*@ lemma lt_tree_node:
    forall x y: key, v: value, l r: tree, c: color.
      lt_tree x l -> lt_tree x r -> y < x -> lt_tree x (Node c l y v r) *)

(*@ lemma gt_tree_node:
    forall x y: key, v: value, l r: tree, c: color.
```

```
      gt_tree x l -> gt_tree x r -> x < y -> gt_tree x (Node c l y v r) *)

(*@ lemma lt_node_lt:
    forall x y: key, v: value, l r: tree, c: color.
      lt_tree x (Node c l y v r) -> y < x *)

(*@ lemma gt_node_gt:
    forall x y: key, v: value, l r: tree, c: color.
      gt_tree x (Node c l y v r) -> x < y *)

(*@ lemma lt_left:
    forall x y: key, v: value, l r: tree, c: color.
      lt_tree x (Node c l y v r) -> lt_tree x l *)

(*@ lemma lt_right:
    forall x y: key, v: value, l r: tree, c: color.
      lt_tree x (Node c l y v r) -> lt_tree x r *)

(*@ lemma gt_left:
    forall x y: key, v: value, l r: tree, c: color.
      gt_tree x (Node c l y v r) -> gt_tree x l *)

(*@ lemma gt_right:
    forall x y: key, v: value, l r: tree, c: color.
      gt_tree x (Node c l y v r) -> gt_tree x r *)

(*@ lemma lt_tree_not_in:
    forall x: key, t: tree. lt_tree x t -> forall v: value. not (memt t x v) *)

(*@ lemma lt_tree_trans:
    forall x y: key. x < y -> forall t: tree. lt_tree x t -> lt_tree y t *)

(*@ lemma gt_tree_not_in:
    forall x: key, t: tree. gt_tree x t -> forall v: value. not (memt t x v) *)

(*@ lemma gt_tree_trans:
    forall x y: key. y < x -> forall t: tree. gt_tree x t -> gt_tree y t *)

(*@ predicate bst (t: tree) =
    match t with
    | Leaf -> true
    | Node _ l k _ r -> bst l && bst r && lt_tree k l && gt_tree k r *)

(*@ lemma bst_Leaf: bst Leaf *)

(*@ lemma bst_left:
    forall k: key, v: value, l r: tree, c: color.
      bst (Node c l k v r) -> bst l *)

(*@ lemma bst_right:
```

```
    forall k: key, v: value, l r: tree, c: color.
      bst (Node c l k v r) -> bst r *)

(*@ lemma bst_color:
    forall c c': color, k: key, v: value, l r: tree.
      bst (Node c l k v r) -> bst (Node c' l k v r) *)

(*@ lemma rotate_left:
    forall kx ky: key, vx vy: value, a b c: tree, c1 c2 c3 c4: color.
      bst (Node c1 a kx vx (Node c2 b ky vy c)) ->
      bst (Node c3 (Node c4 a kx vx b) ky vy c) *)

(*@ lemma rotate_right:
    forall kx ky: key, vx vy: value, a b c: tree, c1 c2 c3 c4: color.
      bst (Node c3 (Node c4 a kx vx b) ky vy c) ->
      bst (Node c1 a kx vx (Node c2 b ky vy c)) *)

(*@ predicate is_not_red (t: tree) =
    match t with
    | Node Red _ _ _ _ -> false
    | Leaf -> true
    | Node Black _ _ _ _ -> true *)

(*@ predicate rbtree (n: int) (t: tree) =
    match t with
    | Leaf -> n = 0
    | Node Red l _ _ r ->
        rbtree n l && rbtree n r && is_not_red l && is_not_red r
    | Node Black l _ _ r ->
        rbtree (n-1) l && rbtree (n-1) r *)

(*@ lemma rbtree_Leaf: rbtree 0 Leaf *)

(*@ lemma rbtree_Node1:
    forall k: key, v: value. rbtree 0 (Node Red Leaf k v Leaf) *)

(*@ lemma rbtree_left:
    forall x: key, v: value, l r: tree, c: color.
      (exists n: int. rbtree n (Node c l x v r)) -> exists n: int. rbtree n l *)

(*@ lemma rbtree_right:
    forall x: key, v: value, l r: tree, c: color.
      (exists n: int. rbtree n (Node c l x v r)) -> exists n: int. rbtree n r *)

exception Not_found

fun find t k =
  (case t of
    Leaf => raise Not_found
  | (Node _ l kqt v r) =>
```

```
    if k = kqt then v
    else if k < kqt then find l k
    else find r k)
(*@ result = find t k
    requires bst t
    variant t
    ensures memt t k result
    raises Not_found -> forall v: value. not (memt t k v) *)

(*@ predicate almost_rbtree (n: int) (t: tree) =
    match t with
    | Leaf -> n = 0
    | Node Red l _ _ r -> rbtree n l /\ rbtree n r
    | Node Black l _ _ r -> rbtree (n-1) l /\ rbtree (n-1) r *)

(*@ lemma rbtree_almost_rbtree:
    forall n: int, t: tree. rbtree n t -> almost_rbtree n t *)

(*@ lemma rbtree_almost_rbtree_ex:
    forall s: tree.
      (exists n: int. rbtree n s) -> exists n: int. almost_rbtree n s *)

(*@ lemma almost_rbtree_rbtree_black:
    forall x: key, v: value, l r: tree, n: int.
      almost_rbtree n (Node Black l x v r) -> rbtree n (Node Black l x v r) *)

fun lbalance l k v r =
  (case l of
    (Node Red (Node Red a kx vx b) ky vy c) =>
      Node Red (Node Black a kx vx b) ky vy (Node Black c k v r)
  | (Node Red a kx vx (Node Red b ky vy c)) =>
      Node Red (Node Black a kx vx b) ky vy (Node Black c k v r)
  | _ => Node Black l k v r)
(*@ result = lbalance l k v r
    requires lt_tree k l && gt_tree k r && bst l && bst r
    ensures bst result &&
      (forall n: int. almost_rbtree n l -> rbtree n r -> rbtree (n+1) result) &&
      forall k': key, v': value.
        memt result k' v' <->
        if k' = k then v' = v else memt l k' v' || memt r k' v' *)

fun rbalance l k v r =
  (case r of
    (Node Red (Node Red b ky vy c) kz vz d) =>
      Node Red (Node Black l k v b) ky vy (Node Black c kz vz d)
  | (Node Red b ky vy (Node Red c kz vz d)) =>
      Node Red (Node Black l k v b) ky vy (Node Black c kz vz d)
  | _ => Node Black l k v r)
(*@ result = rbalance l k v r
    requires lt_tree k l && gt_tree k r && bst l && bst r
```

```
    ensures bst result &&
      (forall n: int. almost_rbtree n r -> rbtree n l -> rbtree (n+1) result) &&
      forall k': key, v': value.
        memt result k' v' <->
        if k' = k then v' = v else memt l k' v' || memt r k' v' *)

fun insert t k v =
  (case t of
    Leaf => Node Red Leaf k v Leaf
  | (Node Red l kqt vqt r) =>
    if k < kqt then Node Red (insert l k v) kqt vqt r
    else if kqt < k then Node Red l kqt vqt (insert r k v)
    else Node Red l kqt v r
  | (Node Black l kqt vqt r) =>
    if k < kqt then lbalance (insert l k v) kqt vqt r
    else if kqt < k then rbalance l kqt vqt (insert r k v)
    else Node Black l kqt v r)
(*@ result = insert t k v
    requires bst t && exists n: int. rbtree n t
    variant t
    ensures bst result &&
      (forall n: int. rbtree n t ->
        almost_rbtree n result &&
        (is_not_red t -> rbtree n result)) &&
      memt result k v &&
      forall k': key, v': value.
        memt result k' v' <->
        if k' = k then v' = v else memt t k' v' *)

exception Absurd

fun add t k v =
  (case insert t k v of
    (Node _ l kqt vqt r) => Node Black l kqt vqt r
  | Leaf => raise Absurd)
(*@ result = add t k v
    requires bst t
    requires exists n: int. rbtree n t
    ensures bst result
    ensures exists n: int. rbtree n result
    ensures memt result k v
    ensures forall k': key, v': value.
    memt result k' v' <-> if k' = k then v' = v else memt t k' v' *)
```

### A.1.4 Generated WhyML Code

```
module RedBlackTree                                                    WhyML

  use int.Int
```

```
 type key = int

 type value = int

 type color =
| Red
| Black

 type tree =
| Leaf
| Node color tree int int tree

 predicate memt (t: tree) (k: key) (v: value) =
match t with
| Leaf -> false
| Node _ l k' v' r -> k = k' /\ v = v' \/ memt l k v \/ memt r k v
 end

 lemma memt_color : forall l: tree, r: tree, k: key, k': key, v: value, v': value, c: color, c': color. (mem

 predicate lt_tree (x: key) (t: tree) =
forall k: key. forall v: value. (memt t k v) -> k < x

 predicate gt_tree (x: key) (t: tree) =
forall k: key. forall v: value. (memt t k v) -> x < k

 lemma lt_leaf : forall x: key. lt_tree x Leaf

 lemma gt_leaf : forall x: key. gt_tree x Leaf

 lemma lt_tree_node : forall x: key, y: key, v: value, l: tree, r: tree, c: color. (lt_tree x l) -> (lt_tree

 lemma gt_tree_node : forall x: key, y: key, v: value, l: tree, r: tree, c: color. (gt_tree x l) -> (gt_tree

 lemma lt_node_lt : forall x: key, y: key, v: value, l: tree, r: tree, c: color. (lt_tree x (Node c l y v r

 lemma gt_node_gt : forall x: key, y: key, v: value, l: tree, r: tree, c: color. (gt_tree x (Node c l y v r

 lemma lt_left : forall x: key, y: key, v: value, l: tree, r: tree, c: color. (lt_tree x (Node c l y v r)) -

 lemma lt_right : forall x: key, y: key, v: value, l: tree, r: tree, c: color. (lt_tree x (Node c l y v r))

 lemma gt_left : forall x: key, y: key, v: value, l: tree, r: tree, c: color. (gt_tree x (Node c l y v r)) -

 lemma gt_right : forall x: key, y: key, v: value, l: tree, r: tree, c: color. (gt_tree x (Node c l y v r))

 lemma lt_tree_not_in : forall x: key, t: tree. (lt_tree x t) -> forall v: value. not memt t x v

 lemma lt_tree_trans : forall x: key, y: key. (x < y) -> forall t: tree. (lt_tree x t) -> lt_tree y t
```

```
 lemma gt_tree_not_in : forall x: key, t: tree. (gt_tree x t) -> forall v: value. not memt t x v

 lemma gt_tree_trans : forall x: key, y: key. (y < x) -> forall t: tree. (gt_tree x t) -> gt_tree y t

 predicate bst (t: tree) =
match t with
| Leaf -> true
| Node _ l k _ r -> bst l /\ bst r /\ lt_tree k l /\ gt_tree k r
 end

 lemma bst_Leaf : bst Leaf

 lemma bst_left : forall k: key, v: value, l: tree, r: tree, c: color. (bst (Node c l k v r)) -> bst l

 lemma bst_right : forall k: key, v: value, l: tree, r: tree, c: color. (bst (Node c l k v r)) -> bst r

 lemma bst_color : forall c: color, c': color, k: key, v: value, l: tree, r: tree. (bst (Node c l k v r)) ->

 lemma rotate_left : forall kx: key, ky: key, vx: value, vy: value, a: tree, b: tree, c: tree, c1: color, c2

 lemma rotate_right : forall kx: key, ky: key, vx: value, vy: value, a: tree, b: tree, c: tree, c1: color, c

 predicate is_not_red (t: tree) =
match t with
| Node Red _ _ _ _ -> false
| Leaf -> true
| Node Black _ _ _ _ -> true
 end

 predicate rbtree (n: int) (t: tree) =
match t with
| Leaf -> n = 0
| Node Red l _ _ r -> rbtree n l /\ rbtree n r /\ is_not_red l /\ is_not_red r
| Node Black l _ _ r -> rbtree (n - 1) l /\ rbtree (n - 1) r
 end

 lemma rbtree_Leaf : rbtree 0 Leaf

 lemma rbtree_Node1 : forall k: key, v: value. rbtree 0 (Node Red Leaf k v Leaf)

 lemma rbtree_left : forall x: key, v: value, l: tree, r: tree, c: color. (exists n: int. rbtree n (Node c l

 lemma rbtree_right : forall x: key, v: value, l: tree, r: tree, c: color. (exists n: int. rbtree n (Node c

 exception Not_found

 let rec find t k
 requires { bst t }
 variant { t }
 ensures { memt t k result }
```

```
 raises { Not_found () -> forall v: value. not memt t k v }
= match t with
 | Leaf -> raise Not_found
 | Node _ l kqt v r -> if k = kqt then v else if k < kqt then find l k else find r k
 end

 predicate almost_rbtree (n: int) (t: tree) =
match t with
| Leaf -> n = 0
| Node Red l _ _ r -> rbtree n l /\ rbtree n r
| Node Black l _ _ r -> rbtree (n - 1) l /\ rbtree (n - 1) r
 end

 lemma rbtree_almost_rbtree : forall n: int, t: tree. (rbtree n t) -> almost_rbtree n t

 lemma rbtree_almost_rbtree_ex : forall s: tree. (exists n: int. rbtree n s) -> exists n: int. almost_rbtree

 lemma almost_rbtree_rbtree_black : forall x: key, v: value, l: tree, r: tree, n: int. (almost_rbtree n (Noc

 let lbalance l k v r
 requires { lt_tree k l /\ gt_tree k r /\ bst l /\ bst r }
 ensures { bst result /\ forall n: int. (almost_rbtree n l) -> (rbtree n r) -> rbtree (n + 1) result /\ fora
= match l with
 | Node Red (Node Red a kx vx b) ky vy c -> Node Red (Node Black a kx vx b) ky vy (Node Black c k v r)
 | Node Red a kx vx (Node Red b ky vy c) -> Node Red (Node Black a kx vx b) ky vy (Node Black c k v r)
 | _ -> Node Black l k v r
 end

 let rbalance l k v r
 requires { lt_tree k l /\ gt_tree k r /\ bst l /\ bst r }
 ensures { bst result /\ forall n: int. (almost_rbtree n r) -> (rbtree n l) -> rbtree (n + 1) result /\ fora
= match r with
 | Node Red (Node Red b ky vy c) kz vz d -> Node Red (Node Black l k v b) ky vy (Node Black c kz vz d)
 | Node Red b ky vy (Node Red c kz vz d) -> Node Red (Node Black l k v b) ky vy (Node Black c kz vz d)
 | _ -> Node Black l k v r
 end

 let rec insert t k v
 requires { bst t /\ exists n: int. rbtree n t }
 variant { t }
 ensures { bst result /\ forall n: int. (rbtree n t) -> almost_rbtree n result /\ (is_not_red t) -> rbtree n
= match t with
 | Leaf -> Node Red Leaf k v Leaf
 | Node Red l kqt vqt r -> if k < kqt then Node Red (insert l k v) kqt vqt r else if kqt < k then Node Red l
 | Node Black l kqt vqt r -> if k < kqt then lbalance (insert l k v) kqt vqt r else if kqt < k then rbalance
 end



 let add t k v
```

```
  requires { bst t }
  requires { exists n: int. rbtree n t }
  ensures { bst result }
  ensures { exists n: int. rbtree n result }
  ensures { memt result k v }
  ensures { forall k': key, v': value. memt result k' v' <-> if k' = k then v' = v else memt t k' v' }
 = match insert t k v with
  | Node _ l kqt vqt r -> Node Black l kqt vqt r
  | Leaf -> absurd
  end

end
```